CHAPTER 2 • SNOWFLAKE FOUNDATIONS ~15 Min

Student

Sir, Chapter 1 gave me the shape of Snowflake - what a cloud data warehouse is, why storage and compute are separate, how to set up an account, use SnowSQL, and navigate the object hierarchy. Now I want to actually understand how it works under the hood - what happens when I run a query, why some queries are fast and others are slow, and how to load real data at production scale.

Claude • AI Assistant

Chapter 2 is where architectural intuition becomes operational fluency. By the end of this chapter you will:

1. Understand Snowflake's **three-layer architecture** in detail - cloud services (brain), query processing (muscle), storage (memory) - and what each layer does for every query you run.
2. Know what **micro-partitions** are, why they matter, and how **partition pruning** determines your query performance and cost.
3. Right-size **virtual warehouses** for your workload - understand the credit-doubling ladder, auto-suspend trade-offs, and when to scale up versus scale out with multi-cluster.
4. Choose the correct **table type** for each dataset - permanent, transient, temporary, or external - and understand the cost and recovery implications of each.
5. Build the universal **data loading pipeline** - stage, file format, COPY INTO - and load anything from CSV to Parquet from anywhere.
6. Handle **errors and validation** in production - ON*ERROR options, VALIDATION*MODE dry-runs, LOAD_HISTORY monitoring, and alerts on silent data loss.

CONCEPT CARD

The **three-layer architecture** is the mental model that unlocks everything else. Every SQL statement you write passes through **cloud services** (parsing, optimization, metadata, transactions), gets dispatched to **query processing** (a virtual warehouse of compute nodes), which reads from **storage** (encrypted columnar micro-partitions on cloud object storage).

Understanding this flow lets you reason about cost, performance, and failure for every workload.

ANALOGY

If Chapter 1 was walking around the construction site and learning the lay of the land, Chapter 2 is picking up the tools and understanding what each one does. The crane (query processing warehouse), the storage yard (micro-partitions), the site office (cloud services) - you now learn what each does, when to use it, and how much it costs to run.

Chapters 3 through 5 will show you advanced techniques with these same tools.

REAL WORLD APPLICATION

Every performance problem, cost overrun, and load failure I have debugged in Snowflake for enterprise clients traces back to a Chapter 2 concept. Wrong warehouse size (2.3), non-sargable WHERE clause

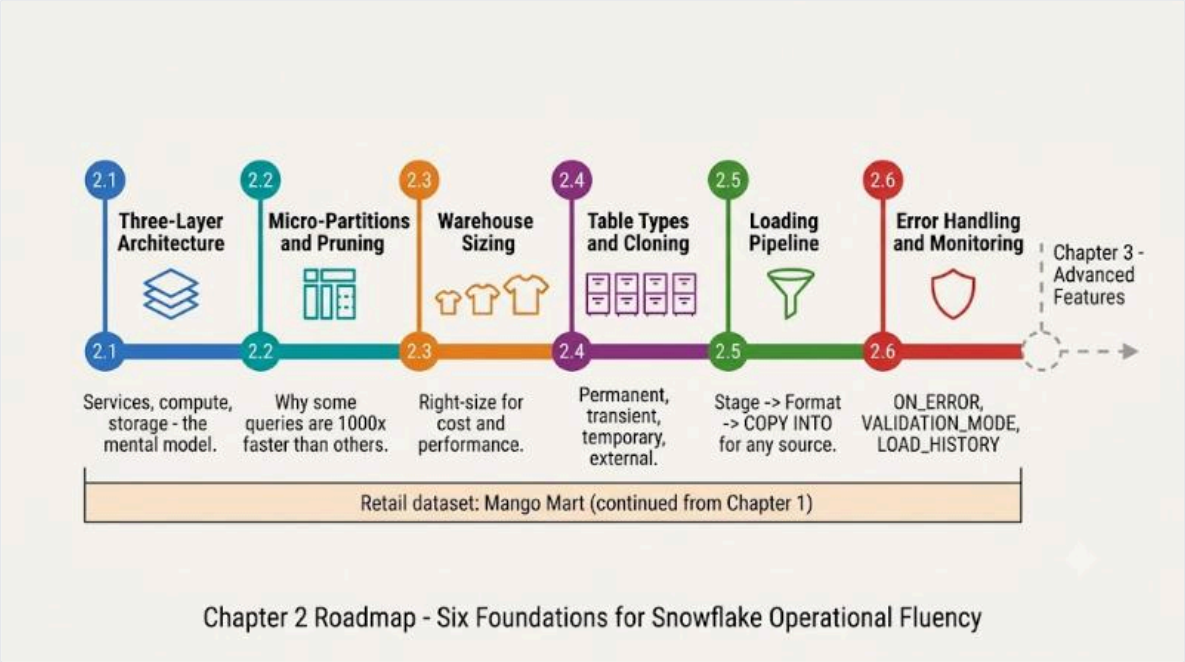
> preventing pruning (2.2), CONTINUE on_error swallowing rejected rows (2.6), permanent tables that should have been transient (2.4).
>
> These are not exotic advanced problems - they are the daily bread of Snowflake operational work. Master this chapter and you can debug 80% of what will ever come up.

Student

Great - let's start with the architecture in depth.

Claude • AI Assistant

Follow me into subsection 2.1.



Chapter 2 Roadmap - Six Foundations for Snowflake Operational Fluency

Chapter 2 at a Glance - Subsection Learning Outcomes

Subsection	Title	Core Question Answered	You Can Now...
2.1	Three-Layer Architecture	What actually happens when I run a query?	Trace a query through services, compute, and storage; predict cost and failure modes.
2.2	Micro-Partitions and Pruning	Why are some queries fast and others slow?	Recognize sargable predicates; read Query Profile for scan efficiency.
2.3	Virtual Warehouse Sizing	How big should my warehouse be, and when does it suspend?	Size warehouses per workload; balance AUTO_SUSPEND against cache warmth.

Subsection	Title	Core Question Answered	You Can Now...
2.4	Database, Schema, Table Types	Which table type for which data?	Choose permanent / transient / temporary / external correctly; use zero-copy cloning.
2.5	Data Loading	How do I load anything from anywhere?	Build the stage -> file format -> COPY INTO pipeline for CSV and Parquet.
2.6	Error Handling and Monitoring	How do I handle bad rows and monitor loads in production?	Choose ON*ERROR correctly; use VALIDATION*MODE; monitor LOAD_HISTORY.

sql — Chapter 2 preview - the mechanisms you will master by the end of the chapter

```sql
-- If you can read every line below at the end of Chapter 2,
-- you are ready for Chapter 3.

-- 2.3 Warehouse per workload
CREATE OR REPLACE WAREHOUSE etl_wh
  WAREHOUSE_SIZE = 'LARGE' AUTO_SUSPEND = 60 AUTO_RESUME = TRUE;

-- 2.4 Transient staging vs permanent analytics
CREATE OR REPLACE TRANSIENT TABLE mango_mart.raw.stg_pos
  (txn_id VARCHAR, sale_ts TIMESTAMP, revenue NUMBER(12,2))
  DATA_RETENTION_TIME_IN_DAYS = 0;

-- 2.5 Stage + File Format + COPY INTO
CREATE OR REPLACE STAGE mango_mart.raw.pos_stage;
CREATE OR REPLACE FILE FORMAT mango_mart.raw.ff_csv
  TYPE = CSV SKIP_HEADER = 1 FIELD_OPTIONALLY_ENCLOSED_BY = '"';

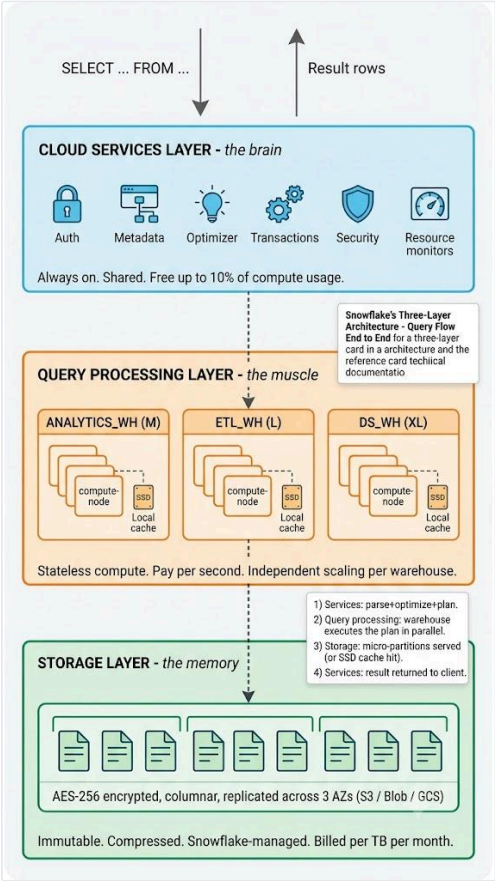
COPY INTO mango_mart.raw.stg_pos
  FROM @mango_mart.raw.pos_stage
  FILE_FORMAT = (FORMAT_NAME = mango_mart.raw.ff_csv)
  ON_ERROR = 'SKIP_FILE_10%';

-- 2.4 Zero-copy clone (dev/test snapshot)
CREATE OR REPLACE TABLE mango_mart.raw.stg_pos_test
  CLONE mango_mart.raw.stg_pos;

-- 2.2 Query Profile - see partition pruning
SELECT partitions_scanned, partitions_total
FROM   TABLE(INFORMATION_SCHEMA.QUERY_HISTORY(RESULT_LIMIT => 5))
ORDER  BY start_time DESC;
```

THREE-LAYER ARCHITECTURE - STORAGE, COMPUTE, AND SERVICES

Claude • AI Assistant



Snowflake's Three-Layer Architecture - Query Flow End to End

sql — Which layer handles what - three sanity queries

```sql
-- Services layer only - no warehouse compute needed.
-- Runs even when your warehouse is suspended.
SHOW TABLES IN SCHEMA mango_mart.retail;
DESCRIBE TABLE mango_mart.retail.dim_product;
SELECT CURRENT_TIMESTAMP(), CURRENT_ACCOUNT();

-- Query processing layer - needs a warehouse.
-- Warehouse auto-resumes if suspended.
SELECT COUNT(*) FROM mango_mart.retail.fact_sales_transaction;

-- Storage layer is invisible to you - never a direct query.
-- But you can see storage usage via services-layer views:
SELECT table_schema, table_name,
       ROUND(active_bytes/POWER(1024,3), 2) AS active_gb,
       row_count
FROM   snowflake.account_usage.table_storage_metrics
```

```
WHERE  table_catalog = 'MANGO_MART'
ORDER  BY active_gb DESC;
```

Student

In Chapter 1 you sketched the three layers - storage, query processing, cloud services. Can we now zoom into each one and understand what actually happens inside?

Claude • AI Assistant

Absolutely. The three-layer architecture is not just a marketing diagram - it is the operational reality of every query you will ever run on Snowflake.

Every SQL statement passes through all three layers in order, and understanding what each layer does lets you reason about performance, cost, and failure modes.

THE CLOUD SERVICES LAYER - THE BRAIN

Student

Start at the top. What is the cloud services layer?

Claude • AI Assistant

CONCEPT CARD

The **cloud services layer** is Snowflake's control plane - a set of always-on services shared across every user and warehouse in your account. It is where your SQL text arrives, gets parsed, optimized, and then dispatched to the query processing layer. Specifically it handles:

- **Authentication and access control** - who you are, which role you are using, what you are allowed to see.
- **Metadata management** - the catalog of every database, schema, table, column, file, and micro-partition. When you run `SHOW TABLES`, this layer answers from metadata without touching storage.
- **Query parsing and optimization** - SQL text becomes a parse tree, then an optimized execution plan. Statistics on micro-partitions drive plan choices.
- **Transaction coordination** - Snowflake uses ACID transactions with multi-version concurrency. This layer keeps the write timeline consistent across all warehouses reading the same table.
- **Security services** - key rotation, encryption, network policies, and audit logging.
- **Infrastructure management** - deciding which warehouse serves a query, sizing suggestions, resource monitors.

Student

Is it a single server?

Claude • AI Assistant

No, it is a set of horizontally scaled, redundant services running across multiple availability zones. From your perspective it looks like a single logical service - you never provision or size it.

Snowflake operates it, patches it, and scales it. You just use it.

This is a genuine software-as-a-service arrangement, unlike traditional 'managed' databases where you still choose an instance size and worry about failover.

THE QUERY PROCESSING LAYER - THE MUSCLE

Student

And the middle layer?

Claude • AI Assistant

CONCEPT CARD

The **query processing layer** is where SQL actually runs. It is composed of **virtual warehouses** - independent MPP compute clusters you create, resize, suspend, and resume on demand. Each warehouse:

- Is a **stateless compute cluster** - if it crashes, the data is safe (in storage) and the metadata is safe (in services); only in-flight query state is lost, and that query can be retried.
- Has a **local SSD cache** on its nodes - the first read of a micro-partition pulls it from remote storage into local SSD; subsequent reads on the same warehouse hit SSD at near-memory speeds.
- Has a **T-shirt size** (XS, S, M, L, XL, 2XL, 3XL, 4XL, 5XL, 6XL) that determines how many compute nodes it has. Each step up doubles the nodes and roughly doubles the cost per hour.
- Can be a **single-cluster warehouse** (one MPP cluster) or a **multi-cluster warehouse** (auto-scaling number of same-sized clusters for concurrency).

Student

What happens when a query arrives?

Claude • AI Assistant

The services layer parses the SQL, checks your permissions, generates an optimized plan, and dispatches the plan to the warehouse you specified with `USE WAREHOUSE` . If the warehouse is suspended, it auto-resumes (typically in under a second on cloud provider capacity).

The warehouse's nodes read the relevant micro-partitions from storage (or from local SSD if cached), execute the plan in parallel, and stream results back to the services layer, which returns them to you.

Student

So the SQL doesn't travel between warehouses?

Claude • AI Assistant

COMMON PITFALL / MISTAKE

Correct - one query runs on exactly one warehouse. If you have three warehouses running three queries concurrently, that is three parallel executions, not one distributed execution.

This is why multi-warehouse setup works for isolation, not for making a single query faster - for that you scale the warehouse UP, not out across warehouses.

THE STORAGE LAYER - THE MEMORY

Student

And storage - is that just S3?

Claude • AI Assistant

CONCEPT CARD

The **storage layer** is Snowflake's abstraction over cloud object storage. What you see as tables is stored as:

- **Micro-partitions** - immutable, columnar-compressed files of 50-500 MB uncompressed (usually 16 MB compressed), one file per table per partition. We cover these in depth in the next subsection.
- **Encrypted at rest** - Snowflake applies AES-256 encryption before writing to the underlying cloud storage. You cannot read the raw files even if you had access to the bucket.
- **Automatically replicated** - the underlying cloud storage (S3, Azure Blob, GCS) replicates across three availability zones for durability.
- **Managed by Snowflake, not by you** - you never see or configure the underlying bucket. From your account's perspective, storage is just 'the data' - measured in TB and billed monthly.

Student

Can I bring my own S3 bucket?

Claude • AI Assistant

For managed tables, no - Snowflake owns and manages the underlying storage. For **external tables** and **Iceberg tables**, yes - you point Snowflake at data in your own S3 / ADLS / GCS bucket and query it in place without loading.

We cover those patterns in Chapter 3. For most workloads, the default managed storage is the right choice - Snowflake handles compression, partitioning, indexing, and replication for you.

HOW THE LAYERS WORK TOGETHER ON ONE QUERY

Student

Walk me through a single query end to end.

Claude • AI Assistant

EXAMPLE CARD

You run:

sql — Code Example

```sql
SELECT category_name, SUM(net_revenue)
FROM   mango_mart.retail.fact_sales_transaction f
JOIN   mango_mart.retail.dim_product p ON f.product_key = p.product_key
WHERE  f.date_key BETWEEN 20241001 AND 20241031
GROUP  BY category_name;
```

1. **Services layer** receives the SQL.

Authenticates your session. Parses the text.

Checks that your role has SELECT on both tables. Builds an execution plan.

Consults micro-partition metadata: 'which partitions of fact*sales*transaction contain rows where date_key is between 20241001 and 20241031?' Metadata says: 3 out of the table's 4,000 partitions.

2. **Query processing layer** - the warehouse you selected wakes up (auto-resume from suspended).

Its nodes are assigned specific micro-partitions to scan. They pull those three partitions from storage (or SSD cache if they were read recently).

They apply the JOIN and GROUP BY in parallel.

3. **Storage layer** - serves the requested micro-partitions. Because only 3 of 4,000 partitions match, the scan touches less than 0.1% of the table. This is the payoff of partition pruning, which we cover next.
4. **Services layer** - collects final results from the warehouse, updates query history, returns rows to you.

Total time for this query on a modest MEDIUM warehouse against a billion-row fact table with good partition pruning: 2-5 seconds.

ANALOGY

Think of the three layers as **a restaurant**. Cloud services = the maitre d' and kitchen manager - takes your order, decides how it will be cooked, coordinates the whole meal.

Query processing = the chefs and cooks - the actual muscle that prepares the food. Storage = the pantry and refrigerators - where all the ingredients are kept, replicated and secured.

You never enter the kitchen. You never touch the pantry.

You just place your order and receive your meal.

REAL WORLD APPLICATION

I once helped a Bengaluru insurance client debug a slow query. The query was 'SELECT COUNT(*) FROM claims'.

It was taking 4 seconds. Turns out the warehouse was cold (just resumed from suspend) and the metadata cache was cold too.

The services layer had to read the partition metadata from persistent storage, then the warehouse had to scan the partition summaries. On the second run: 200 ms - result served entirely from the services-layer metadata cache without touching the warehouse at all.

Understanding the three layers tells you when a query will be cheap (metadata-only) versus expensive (full scan).

WARNING

The three layers are billed separately in principle, but in practice you are billed for two things: **storage** (per TB per month, roughly flat) and **compute credits** (per second per warehouse running). The cloud services layer is free up to 10% of your compute usage - almost all accounts stay well under that.

If you see 'cloud services' charges on your bill, it usually means you are running lots of tiny metadata-only queries (like `SHOW TABLES` in a tight loop) without any warehouse compute to amortize them.

COMMON PITFALL / MISTAKE

Students often think of the warehouse as 'where the data lives'. It is not - the warehouse is pure compute.

If you drop a warehouse, no data is lost. If you resize a warehouse, no data is moved.

The warehouse just reads from the shared storage layer. This decoupling is the entire point of Snowflake's architecture.

EXAM TIP CARD

SnowPro Core will ask you to identify which layer is responsible for a specific function. Memorize: **auth, metadata, optimization, transactions** = services.

SQL execution = query processing. **encrypted micro-partitions on cloud object storage** = storage.

Any question about billing separates 'storage TB' from 'compute credits'.

PRACTICE QUIZ

You run `ALTER WAREHOUSE analytics_wh SET WAREHOUSE_SIZE = 'LARGE';` . Which layer(s) are affected?

A) Only the query processing layer

B) Query processing and storage

C) Services and query processing

D) All three

▼ Reveal Correct Answer

Answer: C. The services layer updates the metadata about the warehouse; the query processing layer resizes the actual cluster. Storage is untouched - no data moves.

EXERCISE

For each operation, list which layers are touched: (a) `SELECT CURRENT_TIMESTAMP()` , (b) `SELECT COUNT(*) FROM huge_table` , (c) `CREATE TABLE new_table (id INT)` , (d) `CREATE WAREHOUSE new_wh WAREHOUSE_SIZE='SMALL'` .

AI DEEP DIVE PROMPT

Ask an AI: '*Explain how a SQL query flows through Snowflake's three architectural layers using a restaurant analogy.*' Compare its answer with the walkthrough above.

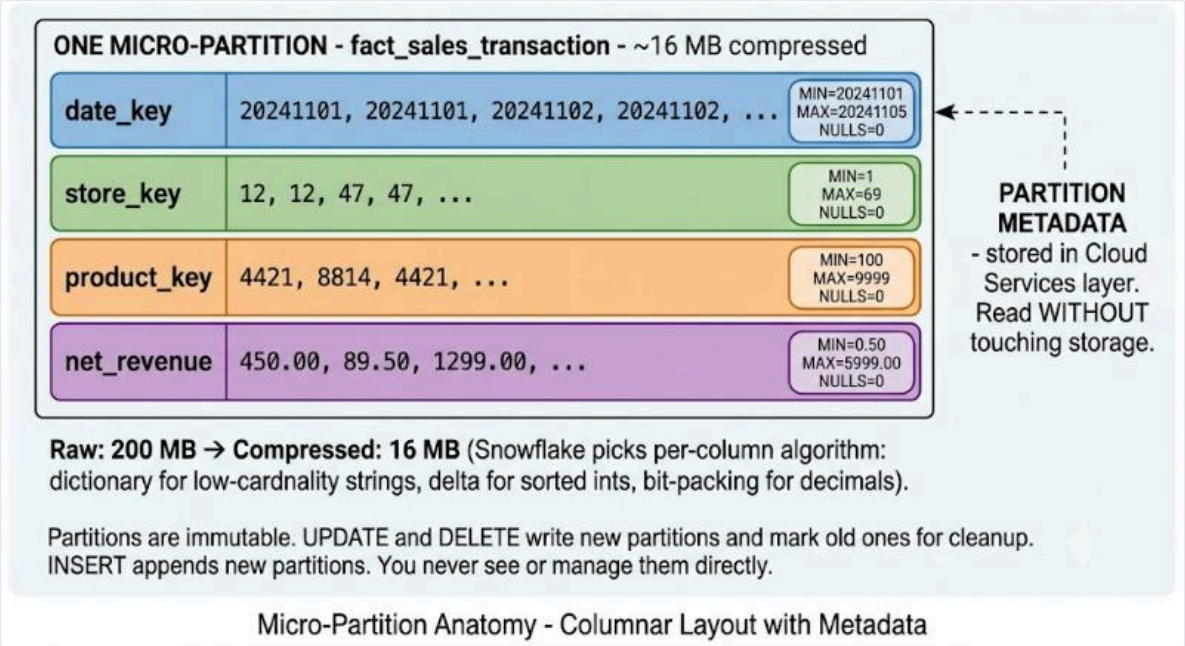
SUMMARY CARD

Snowflake's **three layers** - services (brain), query processing (muscle), storage (memory) - are independent, horizontally scalable, and decoupled. Every query passes through all three.

The services layer is free up to 10% of compute; storage is billed per TB; compute is billed per second per warehouse. Understanding what each layer does lets you predict performance, cost, and failure modes for every workload.

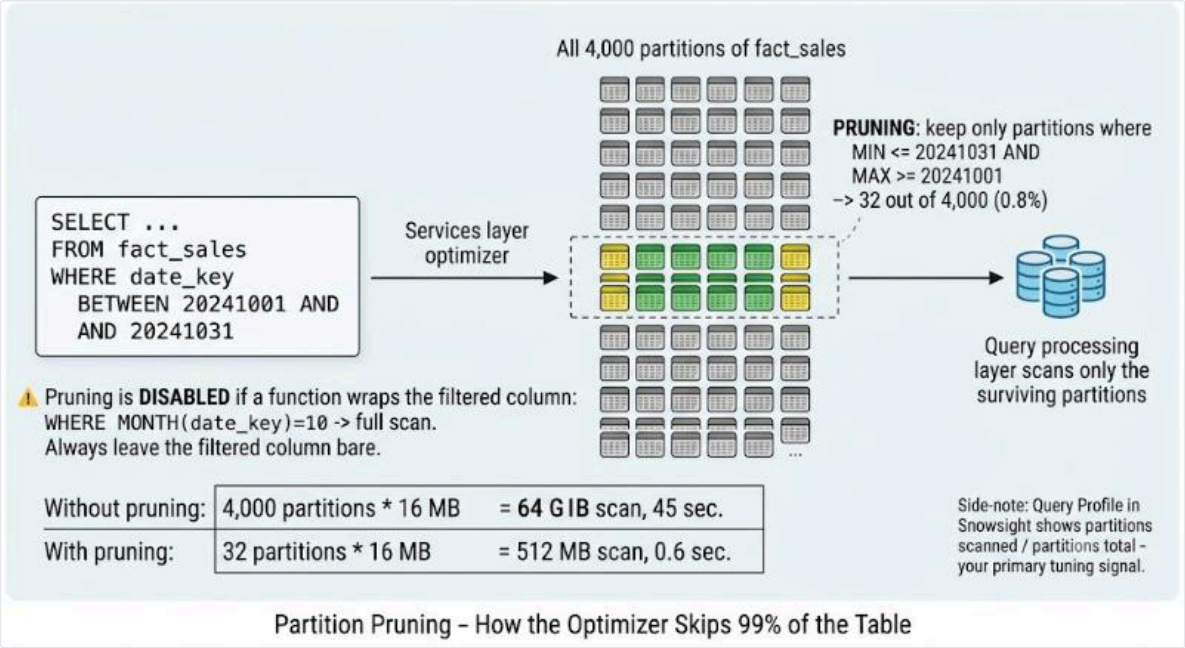
MICRO-PARTITIONS AND PRUNING - HOW SNOWFLAKE FINDS DATA FAST

Claude • AI Assistant



Micro-Partition Anatomy - Columnar Layout with Metadata

Micro-Partition Anatomy - Columnar Layout with Metadata



Partition Pruning – How the Optimizer Skips 99% of the Table

Partition Pruning - How the Optimizer Skips 99% of the Table

sql — See scan efficiency for your recent queries via INFORMATION_SCHEMA

```sql
-- Recent queries and their pruning efficiency
-- Requires INFORMATION_SCHEMA - scoped to the current database.
SELECT
    query_id,
    LEFT(query_text, 80)             AS query_preview,
    warehouse_name,
    total_elapsed_time / 1000        AS seconds,
    partitions_scanned,
    partitions_total,
    ROUND(100.0 * partitions_scanned
               / NULLIF(partitions_total, 0), 1) AS pct_scanned,
    ROUND(bytes_scanned / POWER(1024,3), 2)      AS gb_scanned
FROM   TABLE(INFORMATION_SCHEMA.QUERY_HISTORY(
    RESULT_LIMIT => 20,
    END_TIME_RANGE_START => DATEADD('HOUR', -1, CURRENT_TIMESTAMP())
))
WHERE  execution_status = 'SUCCESS'
  AND  partitions_total > 0
ORDER BY start_time DESC;
```

sql — Sargable vs non-sargable predicates - same result, wildly different scan cost

```sql
-- NON-SARGABLE: function wraps date_key -> full table scan.
-- Pruning is disabled. All 4,000 partitions read.
SELECT COUNT(*) FROM mango_mart.retail.fact_sales_transaction
WHERE TO_VARCHAR(date_key) LIKE '202410%';

-- SARGABLE: bare date_key, plain range comparison -> pruning active.
-- Only October 2024 partitions read (about 32 of 4,000).
SELECT COUNT(*) FROM mango_mart.retail.fact_sales_transaction
```

```
WHERE date_key BETWEEN 20241001 AND 20241031;

-- Even more sargable: half-open range - clearest intent, best pruning.
SELECT COUNT(*) FROM mango_mart.retail.fact_sales_transaction
WHERE date_key >= 20241001 AND date_key < 20241101;
```

Student

You have mentioned micro-partitions a few times. What are they exactly, and why do I need to know?

Claude • AI Assistant

Micro-partitions are the physical unit of storage in Snowflake. Every table is broken into micro-partitions - small, immutable, columnar files - and every query decides which partitions to read based on metadata about each partition.

Understanding this is not optional trivia; it is the mental model that lets you predict query cost and write clustering-friendly ETL.

WHAT IS A MICRO-PARTITION?

Student

Give me the anatomy.

Claude • AI Assistant

CONCEPT CARD

A **micro-partition** is:

- A **single physical file** in Snowflake's managed storage.
- Contains **50 to 500 MB of uncompressed data** (typically 16 MB after Snowflake's columnar compression).
- Stored in **columnar layout** - each column of the rows in that partition is stored contiguously, then compressed independently with the algorithm best suited to that column's data (integers use one algorithm, strings another, timestamps another).
- Holds **metadata** describing the range of values in each column: `MIN` , `MAX` , `DISTINCT_COUNT` , `NULL_COUNT` . This metadata is what makes pruning possible.
- Is **immutable** - once written, it is never modified. Updates and deletes create new partitions and mark the old ones for eventual cleanup.
- Is created **automatically by Snowflake** during any INSERT, COPY INTO, or MERGE operation. You never explicitly create or manage them.

Student

If they are automatic, why do I need to know about them?

Claude • AI Assistant

Because query performance and cost depend directly on how many partitions your query has to scan. A query that reads 3 out of 4,000 partitions is 1,300 times cheaper than a full scan.

The choice to read only 3 is not made by you writing clever SQL - it is made by Snowflake's optimizer, based on the WHERE clause, using the partition metadata. Your job is to write queries and structure ETL so that the optimizer CAN prune.

Knowing the mechanism is what lets you do that.

PARTITION PRUNING - THE PAYOFF

Student

How does the optimizer decide?

Claude • AI Assistant

CONCEPT CARD

Partition pruning works like this:

1. Your query has a WHERE clause: `WHERE date_key = 20241105` .
2. The optimizer looks at each micro-partition's metadata for the `date_key` column: MIN, MAX.
3. Any partition whose MIN > 20241105 or MAX < 20241105 CANNOT contain matching rows. Skip it.
4. Any partition whose MIN <= 20241105 <= MAX MIGHT contain matching rows. Read it.
5. Only the surviving partitions are read from storage. This is called **partition pruning** or **scan pruning**.

The more selective your WHERE clause and the better clustered your data on that column, the fewer partitions survive. A well-pruned query on a 100 TB table might touch only a few GB.

Student

What if my WHERE clause is on a column the data is not clustered on?

Claude • AI Assistant

CONCEPT CARD

Pruning still works, just less effectively. Consider a fact table loaded in date order. `date_key` values are tightly clustered - each partition has a narrow MIN-MAX range on date*key. Pruning on date*key is highly effective.

But `customer_key` might be scattered - each partition has a wide MIN-MAX range on customer*key (many different customers per day of data). Filtering on `customer*key` prunes less effectively - most

partitions might still overlap the filter range.

The result: a customer-key filter reads more partitions than a date-key filter on the same table.

This is where **clustering keys** come in. You can tell Snowflake to physically re-order data by a chosen column (or set of columns), improving pruning for queries on that column. Clustering keys are covered in depth in Chapter 3.

AUTOMATIC (NATURAL) CLUSTERING

Student

You said 'the order they arrived'. Is that always good?

Claude • AI Assistant

For most tables, yes - and this is called **natural clustering**. If your ETL loads yesterday's transactions once a night, all rows for a given `date_key` land in the same batch and end up in the same micro-partitions.

Pruning by date_key works great naturally. You get this without configuring anything.

Natural clustering breaks down in three cases:

- The table is loaded from an unsorted source (e.g., random-order CSVs).
- The table has been heavily UPDATEd and DELETEd, causing 'churn' - new partitions no longer align with the original clustering.
- The queries filter on a different column than the loading order (e.g., loaded by date but queried by customer).

For any of these, you may need an explicit clustering key. Snowflake also runs a background service called **automatic clustering** that re-clusters your table without you needing to trigger it.

QUERY PROFILE - SEEING PRUNING IN ACTION

Student

How do I know if my query pruned well?

Claude • AI Assistant

CONCEPT CARD

Every query you run in Snowsight has a **Query Profile** view showing exactly how the query executed. The key metrics for pruning are:

- **Partitions total** - the total number of partitions in the table.
- **Partitions scanned** - how many the query actually read.
- **Bytes scanned** - actual storage read.

A query that reads 3 of 4,000 partitions shows 'Partitions scanned: 3 / 4000 (0.08%)'. That is excellent pruning.

A query that reads 4,000 of 4,000 is doing a full table scan - either because the WHERE clause is not selective, the data is not clustered on the filtered column, or there is no WHERE clause at all.

Student

Can I see this in SQL, not just the UI?

Claude • AI Assistant

EXAMPLE CARD

Yes - `INFORMATION_SCHEMA.QUERY_HISTORY` and `SYSTEM$CLUSTERING_INFORMATION` expose these details. See the code block for a query that shows scan efficiency for the last 10 queries you ran.

COMPRESSION - THE SECONDARY WIN

Student

How does columnar compression help?

Claude • AI Assistant

When each column is stored contiguously, similar values sit next to each other. A `category_name` column with 5 possible values ('Grocery', 'Dairy', 'Electronics', 'Apparel', 'Stationery') compresses to almost nothing - dictionary encoding stores each unique value once with a small integer reference.

A `net_revenue` column of decimals compresses via bit-packing and delta encoding.

Typical Snowflake compression ratios are 3x to 5x on real-world data. A 100 TB raw dataset lands as 20-30 TB in Snowflake storage - and you pay for the compressed size only. This is a genuine cost saving, transparent to you.

ANALOGY

Micro-partitions with metadata are like **a library with a card catalog**. Each book (partition) has a card showing the range of subjects, authors, and publication dates it covers.

When you ask for 'books on gardening published in 2023', the librarian (optimizer) scans the cards, not the books. She only pulls down the handful of books whose cards say they might match.

The card catalog (metadata) is why the search is fast even when the library has 10 million books.

REAL WORLD APPLICATION

A retail client of mine had a fact_sales table with 8 billion rows spread across 40,000 micro-partitions. Their most-run query was 'yesterday's revenue by region'.

Analysis showed the query was reading 40,000 of 40,000 partitions - a full scan every time. Why?

The WHERE clause was `WHERE DATE(sale_timestamp) = CURRENT_DATE - 1` and `sale_timestamp` was a TIMESTAMP column - the DATE() cast disabled pruning. Rewriting to `WHERE sale_timestamp >= CURRENT_DATE - 1 AND sale_timestamp < CURRENT_DATE` dropped the scan to 4 partitions.

Same result, 10,000x faster. Same query, one small SQL change.

This is the payoff of understanding pruning.

⚠ WARNING

Functions applied to a column in the WHERE clause usually disable pruning on that column. `WHERE DATE(sale_timestamp) = '2024-11-05'` prevents pruning; `WHERE sale_timestamp >= '2024-11-05' AND sale_timestamp < '2024-11-06'` allows it.

This is called **sargability** in SQL parlance and it is a common performance trap.

COMMON PITFALL / MISTAKE

Students see 'partitions' and think of Oracle-style user-managed range partitions. Snowflake micro-partitions are **not user-managed**.

You do not CREATE, ALTER, or reference them explicitly. They are Snowflake's internal storage unit.

What you control is the CLUSTERING KEY - a hint that guides how Snowflake groups new data into partitions.

EXAM TIP CARD

SnowPro Core loves questions about micro-partition size (16 MB compressed, 50-500 MB uncompressed), immutability, and the automatic collection of MIN/MAX metadata. Also expect a question about which WHERE clause allows pruning - always the one WITHOUT a function wrapping the filtered column.

PRACTICE QUIZ

Which WHERE clause allows partition pruning on a `sale_date DATE` column?

A) `WHERE YEAR(sale_date) = 2024`

B) `WHERE sale_date BETWEEN '2024-01-01' AND '2024-12-31'`

C) `WHERE TO_CHAR(sale_date, 'YYYY') = '2024'`

D) `WHERE EXTRACT(YEAR FROM sale_date) = 2024`

▼ Reveal Correct Answer

Answer: B. Only B leaves sale_date unwrapped in the comparison. A, C, and D apply functions that disable pruning.

EXERCISE

Take a query you have written recently. Rewrite it to make every WHERE clause sargable (no function wrapping filtered columns). Run both versions and compare Partitions scanned in the Query Profile.

AI DEEP DIVE PROMPT

Ask an AI: '*Explain Snowflake micro-partitions and pruning using a library card catalog analogy. Give three examples of WHERE clauses that break pruning and how to rewrite them.*'

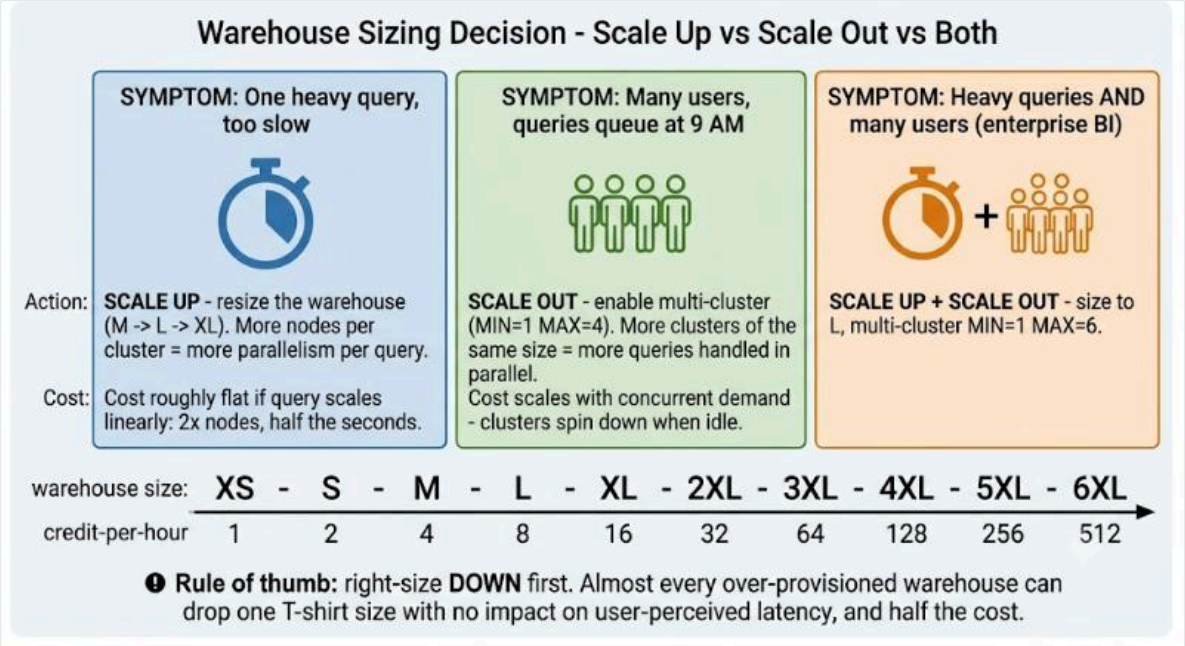
SUMMARY CARD

Micro-partitions are Snowflake's automatic, immutable, columnar, compressed storage units - typically 16 MB compressed, 50-500 MB uncompressed. Each partition carries MIN/MAX/null-count metadata per column, which drives **pruning** - the process of skipping partitions that cannot match a WHERE clause.

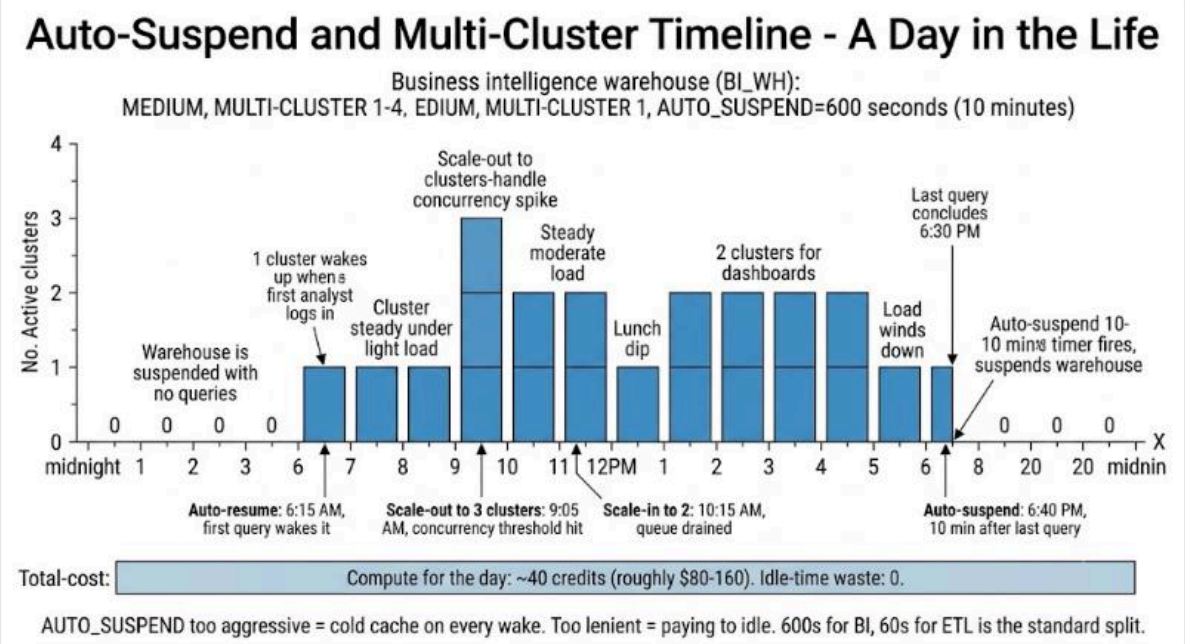
Sargable predicates (no functions on filtered columns) and natural or explicit clustering are how you get maximum pruning. The Query Profile shows exactly how many partitions each query read - your primary performance-tuning signal.

VIRTUAL WAREHOUSES - SIZING, AUTO-SUSPEND, AND MULTI-CLUSTER

Claude • AI Assistant



Warehouse Sizing Decision - Scale Up vs Scale Out vs Both



Auto-Suspend and Multi-Cluster Timeline - A Day in the Life

sql — Create four warehouses for four workloads - the standard production pattern

```sql
-- Nightly ETL: heavy, brief, no concurrency needed
CREATE OR REPLACE WAREHOUSE etl_wh
  WAREHOUSE_SIZE     = 'LARGE'
  AUTO_SUSPEND       = 60
  AUTO_RESUME        = TRUE
  MIN_CLUSTER_COUNT  = 1
  MAX_CLUSTER_COUNT  = 1
  INITIALLY_SUSPENDED = TRUE
  COMMENT = 'Nightly ELT loads - resized larger during month-end.';

-- BI dashboards: moderate size, high concurrency during business hours
CREATE OR REPLACE WAREHOUSE bi_wh
  WAREHOUSE_SIZE     = 'MEDIUM'
  AUTO_SUSPEND       = 600
  AUTO_RESUME        = TRUE
  MIN_CLUSTER_COUNT  = 1
  MAX_CLUSTER_COUNT  = 4
  SCALING_POLICY     = 'STANDARD'
  COMMENT = 'BI dashboards - autoscale out for 9 AM login spike.';

-- Ad-hoc analyst queries: small, low volume, aggressive suspend
CREATE OR REPLACE WAREHOUSE adhoc_wh
  WAREHOUSE_SIZE     = 'SMALL'
  AUTO_SUSPEND       = 60
  AUTO_RESUME        = TRUE
  MIN_CLUSTER_COUNT  = 1
  MAX_CLUSTER_COUNT  = 1
  COMMENT = 'Analyst exploration - cheap, disposable.';

-- Snowpipe / continuous ingestion: tiny, always ready
CREATE OR REPLACE WAREHOUSE load_wh
```

```
  WAREHOUSE_SIZE     = 'XSMALL'
  AUTO_SUSPEND       = 60
  AUTO_RESUME        = TRUE
  COMMENT = 'Snowpipe and continuous loads.';
```

sql — Monitor warehouse cost and utilization - which warehouses earn their keep

```
-- Credits consumed per warehouse over the last 30 days
SELECT
    warehouse_name,
    ROUND(SUM(credits_used), 2)                AS credits_last_30d,
    ROUND(SUM(credits_used) * 3.00, 2)         AS approx_usd_at_3_per_credit,
    ROUND(SUM(credits_used_compute), 2)        AS compute_credits,
    ROUND(SUM(credits_used_cloud_services), 2) AS services_credits
FROM   snowflake.account_usage.warehouse_metering_history
WHERE  start_time >= DATEADD('DAY', -30, CURRENT_TIMESTAMP())
GROUP  BY warehouse_name
ORDER  BY credits_last_30d DESC;

-- Warehouses that are too big for their actual query size
-- (long-running warehouse but small average query duration)
SELECT
    warehouse_name,
    COUNT(*)                                           AS query_count,
    ROUND(AVG(total_elapsed_time / 1000), 1)           AS avg_query_seconds,
    ROUND(SUM(total_elapsed_time / 1000) / 60.0, 1)    AS total_minutes_running,
    CASE WHEN AVG(total_elapsed_time / 1000) < 5
         THEN 'Consider smaller size'
         ELSE 'OK' END                                 AS sizing_hint
FROM   snowflake.account_usage.query_history
WHERE  start_time >= DATEADD('DAY', -7, CURRENT_TIMESTAMP())
  AND  warehouse_size <> 'X-Small'
GROUP  BY warehouse_name
ORDER  BY total_minutes_running DESC;
```

Student

Chapter 1 introduced warehouses and mentioned T-shirt sizes. Now can we go into how to actually pick a size, set auto-suspend, and configure multi-cluster properly?

Claude • AI Assistant

Warehouse sizing is the single biggest lever you have over both cost and performance in Snowflake. A misconfigured warehouse is the number-one cause of runaway bills and slow dashboards.

This subsection covers the four decisions you make for every warehouse: **size** (how big), **auto-suspend** (when to shut off), **auto-resume** (when to wake up), and **multi-cluster** (whether to scale out for

concurrency).

T-SHIRT SIZING - THE LINEAR COST MODEL

Student

Walk me through the sizes.

Claude • AI Assistant

CONCEPT CARD

Warehouse sizes go from **X-Small (XS) to 6XL**. Each step up **doubles the compute nodes** and **doubles the credit-per-hour cost**. The consumption ladder is: XS=1, S=2, M=4, L=8, XL=16, 2XL=32, 3XL=64, 4XL=128, 5XL=256, 6XL=512 credits per hour.

A **credit** is Snowflake's unit of compute billing. Its dollar value depends on your account edition and region - typically
2 – 4percreditforStandard/Enterpriseeditions.SoanXLwarehouserunningnon – stopforonehourcosts16credits, orabout 32-64.

The doubling is the important pattern: going from M to L doubles the cost AND doubles the compute power available to a single query. If your query on M takes 40 seconds, it should take about 20 on L - and cost roughly the same total credits (double per-second rate, half the seconds).

This is the **cost-flat scaling** property: for a query that scales linearly, upsizing doesn't change your bill; it just returns the answer faster.

Student

So why not always run on 6XL?

Claude • AI Assistant

COMMON PITFALL / MISTAKE

Two reasons. First, **not all queries scale linearly**.

A tiny query on XS takes 2 seconds. On 6XL it takes...

2 seconds. The overhead of setting up hundreds of nodes for a trivial workload eats any parallelism benefit.

You paid 512x the credit rate to run for the same wall time.

Second, **auto-suspend and minimum billing** matter. Every warehouse has a **60-second minimum billing** when it runs.

If you run five 2-second queries on 6XL with a 5-minute idle gap, each query triggers a fresh 60-second billing window at 512 credits/hour rate. You paid 60 seconds *5 queries* 512/3600 = 43 credits ($86-170) for 10 seconds of actual work.

On XS: same queries would cost 5 *60* 1/3600 = 0.08 credits (about $0.20). Right-sizing matters enormously.

AUTO-SUSPEND AND AUTO-RESUME

Student

What are auto-suspend and auto-resume?

Claude • AI Assistant

CONCEPT CARD

Auto-suspend is the number of seconds of idle time after which the warehouse stops (and stops billing). **Auto-resume** is a boolean: if TRUE, the next query automatically wakes the warehouse. Configuration:

- `AUTO_SUSPEND = 60` - suspend after 60 seconds of idle. Aggressive - good for interactive workloads.
- `AUTO_SUSPEND = 300` - 5 minutes. Balanced - allows short pauses between queries without paying to restart.
- `AUTO_SUSPEND = 3600` - 1 hour. Conservative - keeps the local SSD cache warm for busy dashboards.
- `AUTO_RESUME = TRUE` - default. When a query arrives, the warehouse wakes up (usually in under a second).

Setting `AUTO_SUSPEND = 0` means 'never auto-suspend' - a very expensive choice you should almost never make. There is also a lower bound: Snowflake enforces a minimum auto-suspend of 60 seconds (you cannot go below this).

Student

What is the trade-off?

Claude • AI Assistant

Two forces pull in opposite directions.

- **Shorter auto-suspend** = lower idle-time waste. Good.
- **Shorter auto-suspend** = colder cache when the warehouse resumes. The first query after resume must re-read micro-partitions from storage (a few seconds slower). Bad.

The rule of thumb:

- **Interactive analyst workloads** - AUTO_SUSPEND = 60. Cache warmth is less important than not paying for idle time between long thinking pauses.
- **BI dashboards refreshing every 5-10 minutes** - AUTO_SUSPEND = 600 (10 min). Keep cache warm across refreshes.

- **Nightly ETL jobs** - AUTO_SUSPEND = 60. The next run is tomorrow anyway - cache warmth is worthless.
- **Multi-tenant SaaS platforms serving user queries all day** - AUTO_SUSPEND = 1800 (30 min) or higher. Cache is highly valuable.

MULTI-CLUSTER WAREHOUSES - SCALE OUT FOR CONCURRENCY

Student

What about multi-cluster?

Claude • AI Assistant

CONCEPT CARD

A **multi-cluster warehouse** is a single logical warehouse (one name, one identity) that can run **multiple identical clusters** in parallel to handle concurrent queries. You configure it with:

- `MIN_CLUSTER_COUNT = 1` - always have at least 1 cluster running when the warehouse is active.
- `MAX_CLUSTER_COUNT = 4` - can auto-scale up to 4 clusters when queries queue.
- `SCALING_POLICY = 'STANDARD'` - eager: add a cluster the moment queries start queuing. Best for interactive workloads.
- `SCALING_POLICY = 'ECONOMY'` - conservative: try to keep the existing clusters busy before adding more. Best for batch workloads where a small wait is acceptable.

Each cluster is the same size as the warehouse. A MEDIUM multi-cluster warehouse with max=4 can spawn up to 4 *MEDIUM = 4* 4 credits/hour = 16 credits/hour at peak. When queries subside, clusters spin down automatically.

Student

When do I use multi-cluster versus just a bigger single cluster?

Claude • AI Assistant

CONCEPT CARD

The **scale-up vs scale-out** decision from Chapter 1:

- **Scale UP** (bigger single cluster: M -> L -> XL) when the bottleneck is **a single heavy query**. More nodes = more parallelism inside one query.
- **Scale OUT** (multi-cluster: 1 cluster -> 4 clusters at same size) when the bottleneck is **many concurrent users**. Each cluster serves a share of the queries; no query gets faster individually, but no query has to wait in a queue.
- Both together for enterprise dashboards: MEDIUM warehouse with multi-cluster 1-4 handles moderate individual queries AND up to 4x concurrent load.

WAREHOUSE-PER-WORKLOAD PATTERN

Student

Should I have one warehouse for everything or one per team?

Claude • AI Assistant

The standard production pattern is **one warehouse per workload**, not per team or per user. Group workloads by their compute needs:

- `ETL_WH` - LARGE, single-cluster, AUTO_SUSPEND=60. Runs nightly ELT jobs. Heavy but rare.
- `BI_WH` - MEDIUM, multi-cluster 1-4, AUTO_SUSPEND=600. Serves BI dashboards during business hours. High concurrency.
- `ADHOC_WH` - SMALL, single-cluster, AUTO_SUSPEND=60. Analyst ad-hoc queries. Low volume, sporadic.
- `LOAD_WH` - XSMALL, single-cluster, AUTO_SUSPEND=60. Snowpipe and continuous ingestion. Constant small work.

This isolation gives you three benefits:

1. **Cost attribution** - you can see exactly how much each workload costs (via QUERY*HISTORY and WAREHOUSE*METERING_HISTORY).
2. **Performance isolation** - the ETL job at 2 AM cannot starve the BI dashboards at 9 AM because they use different warehouses.
3. **Tuning independence** - you can resize BI*WH without affecting ETL*WH.

ANALOGY

Warehouse sizing is like **picking a vehicle for a trip**. XS is a scooter - cheap, one person, fine for a corner errand.

M is a family sedan - handles most needs. XL is a delivery truck - carries heavy loads.

6XL is a semi-truck - only for moving buildings. **Auto-suspend** is turning off the engine at red lights - saves fuel but takes a moment to restart.

Multi-cluster is running a taxi fleet instead of one taxi - each customer gets a car; no one waits. You would not commute in a semi-truck; you would not move a house on a scooter.

Match the vehicle to the trip.

REAL WORLD APPLICATION

A Hyderabad SaaS startup I advised had one warehouse called `MAIN` set to XL, AUTO_SUSPEND=3600, no multi-cluster. It ran everything - ETL, dashboards, ad-hoc queries.

Bill: $18,000/month. We split into four warehouses (ETL*WH=L, BI*WH=M with 1-4 clusters, ADHOC*WH=S, LOAD*WH=XS), each with AUTO_SUSPEND=60 to 600 appropriate to its workload.

Bill next month: $6,200. Same queries, same latency, one-third the cost.

Warehouse hygiene is real money.

WARNING

The MINIMUM 60-second billing per resume is the single most common source of surprise costs. If a scheduler triggers 100 tiny queries per hour on a big warehouse with short auto-suspend, you may be paying 100 * 60 seconds = 100 minutes/hour of compute for a few seconds of actual work.

Either batch the queries, use a smaller warehouse, or bump AUTO_SUSPEND up.

COMMON PITFALL / MISTAKE

Setting `AUTO_SUSPEND = 0` because 'my dashboard needs to be instant.' The dashboard is fine with a 1-second warehouse resume - users don't notice. What you notice is the bill: paying for 24/7 XL warehouse = ~11,500 credits/month ~= $23,000-46,000 per warehouse.

EXAM TIP CARD

SnowPro Core will test: (1) the credit-doubling ladder (XS=1, M=4, XL=16), (2) the 60-second minimum billing after resume, (3) scale-up vs scale-out distinction, (4) STANDARD vs ECONOMY scaling policy, (5) minimum AUTO_SUSPEND is 60 seconds.

PRACTICE QUIZ

Your data science team runs one large training query per day on a MEDIUM warehouse. It takes 90 minutes. They complain it is too slow. Best action?

A) Add multi-cluster max=4

B) Resize the warehouse to XL

C) Set AUTO_SUSPEND=0

D) Move to Business Critical edition

▼ Reveal Correct Answer

Answer: B. Single heavy query needs scale-up (bigger cluster), not scale-out (more clusters). Multi-cluster helps concurrency, not one big query. AUTO_SUSPEND=0 wastes money without speeding up. Edition doesn't affect this.

EXERCISE

Design warehouse configurations for Mango Mart's three teams: (a) 20 analysts running ad-hoc queries throughout the day, (b) nightly ETL of 500 GB of new data, (c) a public marketing dashboard hit by ~200 concurrent users at 9 AM daily. Specify size, AUTO_SUSPEND, and multi-cluster settings for each.

AI DEEP DIVE PROMPT

Ask an AI: '*Design a Snowflake warehouse sizing strategy for a mid-sized e-commerce company with three teams: nightly ETL, BI dashboards used all day by 100 people, and a small data science team running weekly ML training. Include AUTO_SUSPEND settings and estimated monthly cost.*'

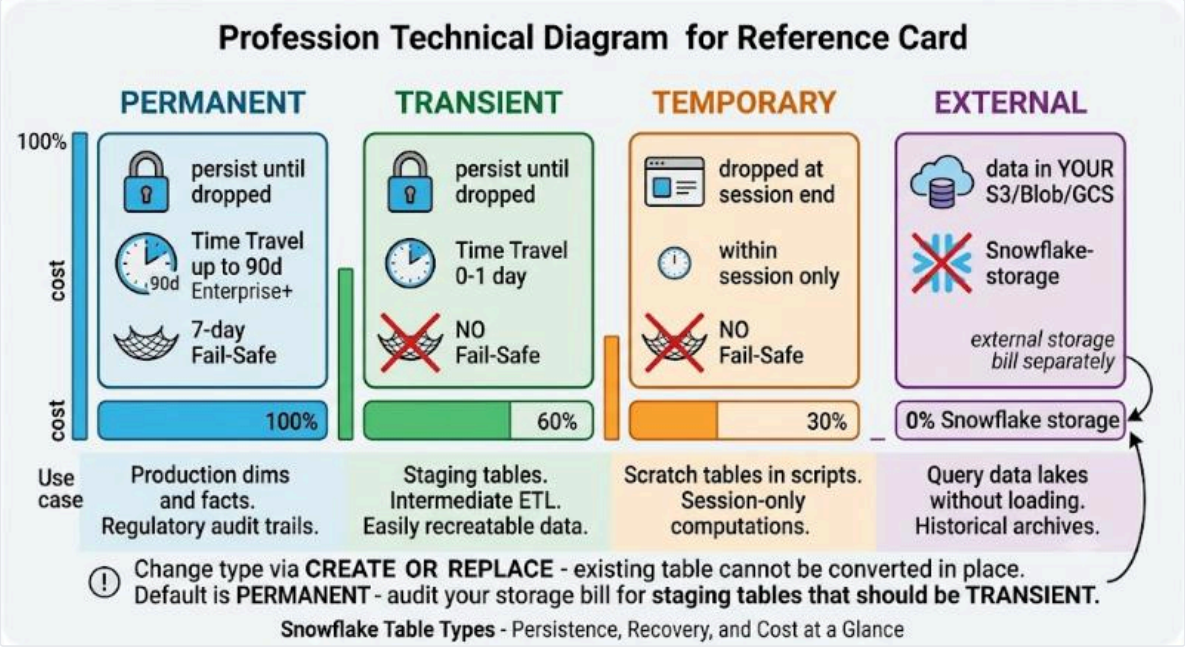
SUMMARY CARD

Warehouse sizes double in cost and compute at each step (XS=1 to 6XL=512 credits/hour). **Auto-suspend** trades cache warmth for idle-time savings; typical values are 60s for ETL, 600s for BI.

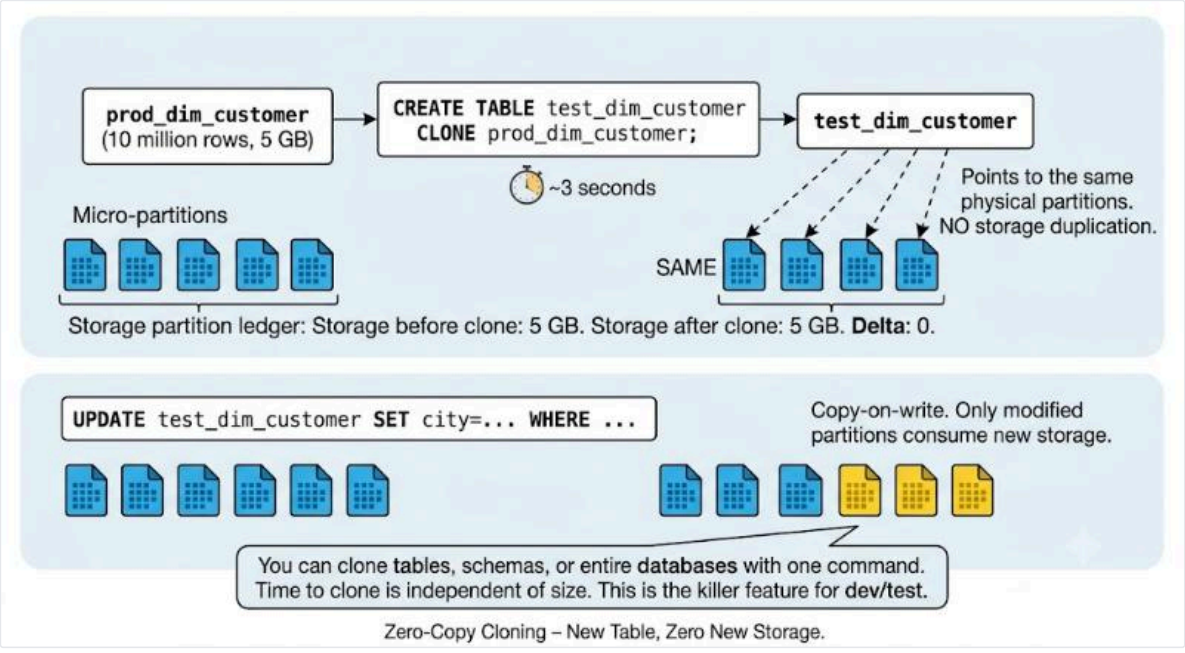
> **Multi-cluster** scales out for concurrency; MIN*CLUSTER*COUNT=1 MAX*CLUSTER*COUNT=4 with STANDARD scaling is a good default. The production pattern is **one warehouse per workload**, not per team - giving you cost attribution, performance isolation, and independent tuning.

DATABASES, SCHEMAS, AND TABLE TYPES - PERMANENT, TRANSIENT, TEMPORARY

Claude • AI Assistant



Snowflake Table Types - Persistence, Recovery, and Cost at a Glance



Zero-Copy Cloning - New Table, Zero New Storage

```
sql — Create tables of all four types - and see the retention setting explicitly
```

```
-- PERMANENT (default): full Time Travel + Fail-Safe.
CREATE OR REPLACE TABLE dim_product (
  product_key      INT PRIMARY KEY,
  product_code     VARCHAR(20) NOT NULL,
  product_name     VARCHAR(200)
)
DATA_RETENTION_TIME_IN_DAYS = 30  -- 30 days of Time Travel
COMMENT = 'Production product dimension.';

-- TRANSIENT: persistent but no Fail-Safe, lower storage cost.
CREATE OR REPLACE TRANSIENT TABLE stg_customer_dedupe (
  customer_id      VARCHAR(20),
  email_normalized VARCHAR(200),
  first_seen       TIMESTAMP_NTZ
)
DATA_RETENTION_TIME_IN_DAYS = 0   -- no Time Travel either
COMMENT = 'Intermediate ETL staging - rebuilt nightly.';

-- TEMPORARY: dropped at session end, invisible to other sessions.
CREATE OR REPLACE TEMPORARY TABLE tmp_customer_ids AS
SELECT DISTINCT customer_key
FROM   fact_sales_transaction
WHERE  date_key BETWEEN 20241001 AND 20241031;

-- EXTERNAL: query data in YOUR S3 bucket without loading.
-- (Requires a Storage Integration and Stage - see next subsection.)
CREATE OR REPLACE EXTERNAL TABLE ext_pos_archive (
  raw_data VARIANT AS (value)
)
  WITH LOCATION = @s3_pos_archive_stage/2023/
  FILE_FORMAT   = (TYPE = PARQUET)
  COMMENT       = 'Cold POS archive - queried a few times a year.';
```

sql — Zero-copy cloning - snapshot a table in seconds without duplicating storage

```
-- Clone a single table for testing.
CREATE OR REPLACE TABLE dim_customer_test
  CLONE dim_customer;

-- Clone an entire schema (all tables, views, sequences, stages).
CREATE OR REPLACE SCHEMA retail_test
  CLONE retail;

-- Clone an entire database (production snapshot in seconds).
CREATE OR REPLACE DATABASE mango_mart_pre_upgrade
  CLONE mango_mart;

-- Point-in-time clone (Time Travel + Clone combined) -
-- snapshot the table as it was 2 hours ago.
CREATE OR REPLACE TABLE dim_customer_snapshot_2h_ago
  CLONE dim_customer AT (OFFSET => -60 * 60 * 2);
```

```
-- Undrop a table you accidentally dropped (within Time Travel window).
UNDROP TABLE dim_customer;

-- Check current storage consumption per table.
SELECT table_schema, table_name, table_type,
       ROUND(active_bytes    / POWER(1024,3), 2) AS active_gb,
       ROUND(time_travel_bytes / POWER(1024,3), 2) AS tt_gb,
       ROUND(failsafe_bytes   / POWER(1024,3), 2) AS fs_gb
FROM   snowflake.account_usage.table_storage_metrics
WHERE  table_catalog = 'MANGO_MART' AND deleted = FALSE
ORDER  BY active_gb DESC;
```

Student

The object hierarchy from Chapter 1 said DATABASE -> SCHEMA -> TABLE. Now I want to go deeper. Are there different kinds of databases, and are all tables the same?

Claude • AI Assistant

Two important refinements. Databases and schemas mostly work the same as in any relational system, but Snowflake adds a few conveniences (managed access schemas, cloning, information schema).

Tables come in **four types** - permanent, transient, temporary, and external - each with different persistence, cost, and recovery characteristics. Choosing the wrong type costs you money on staging tables or, worse, loses data on production tables.

DATABASES - ONE PER LOGICAL DOMAIN

Student

What is the rule of thumb for creating databases?

Claude • AI Assistant

CONCEPT CARD

A **database** in Snowflake is a logical container for one broad domain of data. Typical production accounts have several:

- `RAW` - untransformed data straight from source systems (POS exports, CRM dumps).
- `STAGING` - intermediate tables during ETL/ELT transformations.
- `ANALYTICS` - the presentation layer: cleaned, modeled star schemas ready for BI.
- `SANDBOX` - a per-analyst play area for exploration.
- `PROD_APP` - if you have an operational app reading Snowflake, its own database.

Some organizations use a **medallion architecture** and name them `BRONZE`, `SILVER`, `GOLD` for raw / conformed / analytics-ready. Same idea, different vocabulary.

The point is separation of concerns - RBAC, resource monitors, and retention policies can be different per database.

Student

When would I create multiple databases versus one database with multiple schemas?

Claude • AI Assistant

Databases are the coarser grouping. Use separate databases when:

- The data has different **retention or compliance requirements** (RAW keeps 90-day time travel; STAGING keeps 0 days).
- Different **security domains** (production vs sandbox - even different roles should own them).
- Different **billing centers** (separate cost tracking).

Use schemas within one database when:

- Content is closely related (all belongs to the same domain).
- Same roles and retention apply.
- You want easy cross-schema references without three-part names all over your SQL.

SCHEMA TYPES - A SMALL BUT USEFUL DISTINCTION

Student

Are there different schema types too?

Claude • AI Assistant

CONCEPT CARD

Snowflake has two schema variants:

- **Regular schema** (default) - permissions inherited from the database owner but individually grantable on each object.
- **Managed access schema** (`WITH MANAGED ACCESS`) - only the schema owner can grant privileges on objects in the schema, even if you own the individual objects. Enforces centralized permission control - useful in regulated environments.

Almost all schemas you create will be regular. Managed access schemas are worth knowing about for compliance-heavy workloads.

THE FOUR TABLE TYPES

Student

Now the important part - the four table types.

Claude • AI Assistant

CONCEPT CARD

Permanent tables are the default. `CREATE TABLE foo (...)` creates a permanent table. They:

- **Persist until explicitly dropped** with `DROP TABLE` .
- Have **Time Travel** - retention up to 1 day on Standard edition, up to 90 days on Enterprise+. During this window, you can query the table 'as it was' at any prior timestamp with `AT(TIMESTAMP => ...)` , and you can undrop the table with `UNDROP TABLE foo;` .
- Have a **7-day Fail-Safe** period after Time Travel expires. Fail-Safe is not user-queryable but allows Snowflake support to recover data in disasters.
- Charge you for storage of all the above - active data plus time travel history plus fail-safe.

Student

What are transient tables for?

Claude • AI Assistant

CONCEPT CARD

Transient tables are Snowflake's answer to: 'I need a table that persists across sessions, but I don't need Fail-Safe.' Created with `CREATE TRANSIENT TABLE` . They:

- Persist until dropped (same as permanent).
- Have Time Travel of **0 or 1 day** (configurable), maximum 1 day - not 90.
- Have **NO Fail-Safe** - once time travel expires and you drop, data is truly gone.
- Cost less to store because Snowflake does not maintain the 7-day fail-safe copy.

Perfect for **staging tables** in ETL pipelines. If today's staging table gets corrupted, you re-run the ETL - the source data is elsewhere.

You don't need Fail-Safe to insure against catastrophic loss because the data is recreatable from source. Transient staging tables can save 30-50% on staging-layer storage costs at scale.

Student

And temporary tables?

Claude • AI Assistant

CONCEPT CARD

Temporary tables exist only for the current session. Created with `CREATE TEMPORARY TABLE` (or the shorthand `CREATE TEMP TABLE`). They:

- **Dropped automatically at session end** - when you close your SnowSQL connection, the tables disappear.
- Have no Time Travel beyond the session and no Fail-Safe.
- Are **invisible to other sessions** - even the same user in another session cannot see them.
- Cost the least - Snowflake does not retain data beyond the session.

Perfect for **scratch tables in a multi-step SQL script**. You write intermediate results to a temp table, join it later in the script, and never worry about cleanup - the session end cleans up for you.

Student

And external tables - I have heard the term but do not really understand it.

Claude • AI Assistant

CONCEPT CARD

External tables point at data files in **your own cloud storage** (S3, Azure Blob, GCS) that Snowflake reads without loading. The table's metadata lives in Snowflake, but the data files stay in your bucket. They:

- Have **no Snowflake storage cost** - only the metadata is in Snowflake.
- Have **no Time Travel and no Fail-Safe** - data lifecycle is managed by your cloud storage.
- Are typically slower than native tables because there is no columnar Snowflake format and no fine-grained pruning.
- Are perfect for **querying data lakes** - historical archives you rarely touch, semi-structured logs, or files produced by non-Snowflake systems.

Iceberg tables are a modern evolution of external tables using the Apache Iceberg open table format - Snowflake can read AND write these with full ACID transactions. We cover Iceberg in Chapter 3.

CLONING - THE KILLER FEATURE OF SNOWFLAKE TABLES

Student

Wait - if I want to make a copy of a production table for testing, do I have to duplicate the storage?

Claude • AI Assistant

No - Snowflake has **zero-copy cloning**, one of its most beloved features. `CREATE TABLE test_dim_customer CLONE prod_dim_customer;` creates a new table that shares the underlying micro-partitions with the source.

It takes seconds regardless of the source table's size (10 GB or 100 TB - same speed). The clone is fully writable; when you modify it, only the changed micro-partitions become 'new' and consume additional

storage.

This is enormous for:

- **Dev/test environments** - clone prod to test, modify, drop. Storage stays flat.
- **A/B testing** - clone before applying a migration, test, roll back or forward.
- **Point-in-time backups** - clone at midnight; if the day goes bad, the clone is your snapshot.

You can clone tables, schemas, and even entire databases with a single command. We cover cloning in depth in Chapter 3.

ANALOGY

The table types are like **types of cabinet drawers in a lab**. Permanent tables are the archive drawer - stuff you must keep for years, with backups and versioning.

Transient tables are the working drawer - stuff you use daily but could recreate from raw materials. Temporary tables are the workbench - stuff you have out for one session; the cleaner throws away whatever is on it overnight.

External tables are the offsite warehouse - inventoried on your list but physically stored elsewhere.

REAL WORLD APPLICATION

A client's staging schema had 47 tables, all permanent, all with 90-day time travel. Their monthly storage bill was $4,200 - and their staging data was rebuilt fresh each day.

Changing all 47 tables to TRANSIENT with `DATA_RETENTION_TIME_IN_DAYS = 0` (no time travel, no fail-safe) dropped the bill to $180. Same functionality, same recoverability (they always re-run ETL from source), one-twentieth the cost.

Table type is not academic - it is your storage bill.

WARNING

Dropping a **temporary table** does not require confirmation and there is no UNDROP - session end already deleted it. Dropping a **transient table** has 0 or 1 day of Time Travel and no Fail-Safe - if you set `DATA_RETENTION_TIME_IN_DAYS = 0` and drop, it is gone.

Only **permanent tables** on Enterprise+ give you 90 days of undo + 7 days of Fail-Safe. Choose your type based on how disastrous accidental loss would be.

COMMON PITFALL / MISTAKE

Creating everything as PERMANENT because it is the default. Perfectly valid - and expensive if you have hundreds of staging tables that never needed 90-day time travel. Look at your storage bill; if it seems high, audit which tables really need permanent status.

EXAM TIP CARD

SnowPro Core will test: (1) permanent = default, with Time Travel + Fail-Safe. (2) Transient = persistent, no Fail-Safe.

(3) Temporary = session-scoped, dropped at session end. (4) External = data in your own storage, only metadata in Snowflake.

Also expect a question on `DATA_RETENTION_TIME_IN_DAYS` - the setting that controls time travel duration per table (0 disables it).

PRACTICE QUIZ

You need a staging table that holds intermediate ETL results, persists between the extract and load steps of a nightly job, but need not be recoverable if lost. Which table type?

A) Permanent B) Transient C) Temporary D) External

▼ Reveal Correct Answer

Answer: B. Persists between steps (rules out Temporary - which dies at session end). Need not be recoverable (rules out Permanent's Fail-Safe overhead). Data in Snowflake, not in your own bucket (rules out External).

EXERCISE

Design the table type strategy for Mango Mart: (a) `raw_pos_transactions` (nightly load of yesterday's POS extract), (b) `stg_customer_dedupe` (intermediate deduplication step in customer ETL), (c) `fact_sales_transaction` (the analytics table), (d) `session_temp_customer_list` (a scratch table in a single SnowSQL script). Choose PERMANENT / TRANSIENT / TEMPORARY / EXTERNAL for each and justify.

AI DEEP DIVE PROMPT

Ask an AI: '*Explain the four Snowflake table types (permanent, transient, temporary, external) with a use case for each and their storage cost implications.*'

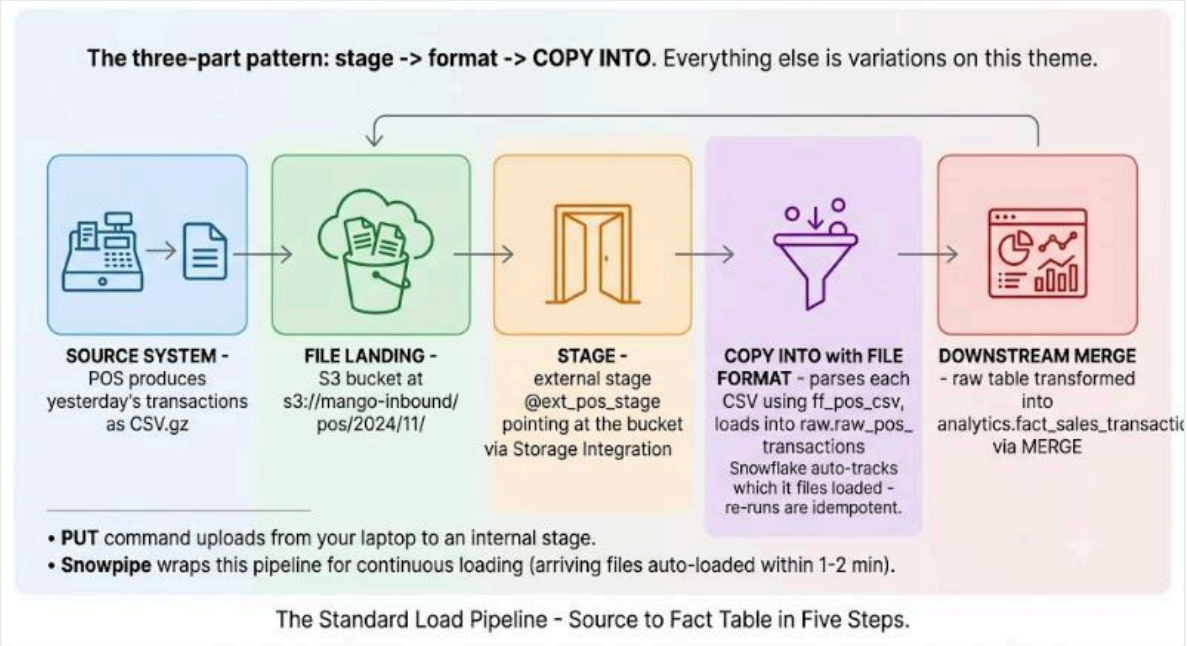
SUMMARY CARD

Snowflake has **four table types**: **permanent** (default, full Time Travel + Fail-Safe), **transient** (persistent, no Fail-Safe, lower cost), **temporary** (session-only, cheapest), and **external** (data in your own cloud storage). Databases separate broad domains (RAW, STAGING, ANALYTICS); schemas group related tables within a database.

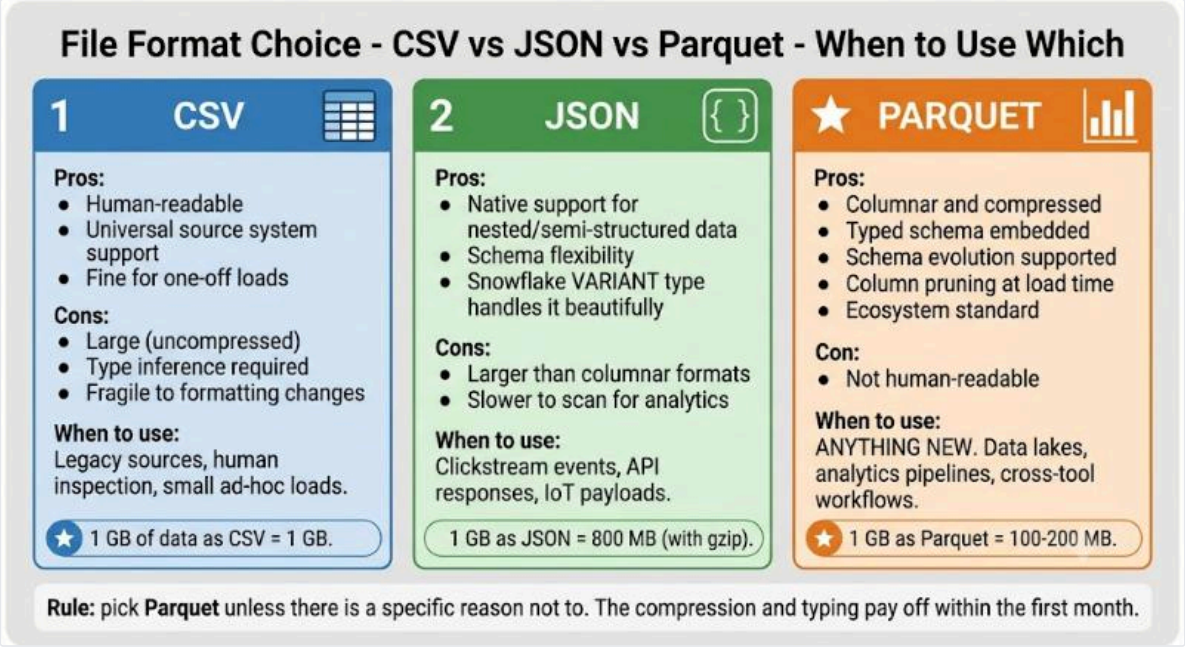
Zero-copy cloning creates writable snapshots in seconds without duplicating storage - the killer feature for dev/test workflows.

DATA LOADING - STAGES, FILE FORMATS, AND COPY INTO

Claude • AI Assistant



The Standard Load Pipeline - Source to Fact Table in Five Steps



File Format Choice - CSV vs JSON vs Parquet - When to Use Which

sql — Create the loading infrastructure - integration, stage, file format, then load

```sql
-- Storage Integration: one-time setup granting Snowflake access to your S3
CREATE OR REPLACE STORAGE INTEGRATION si_mango_inbound
  TYPE = EXTERNAL_STAGE
  STORAGE_PROVIDER = 'S3'
  ENABLED = TRUE
  STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::123456789012:role/snowflake_read'
  STORAGE_ALLOWED_LOCATIONS = ('s3://mango-inbound/');

-- Note: DESCRIBE the integration to get the External ID and AWS role
-- that must be granted trust in AWS IAM - one-time setup.
```

```
DESCRIBE INTEGRATION si_mango_inbound;

-- External stage: points at the S3 prefix via the integration.
CREATE OR REPLACE STAGE ext_pos_stage
  URL = 's3://mango-inbound/pos/'
  STORAGE_INTEGRATION = si_mango_inbound
  COMMENT = 'Landing zone for nightly POS extracts.';

-- File format: how to parse the POS CSVs.
CREATE OR REPLACE FILE FORMAT ff_pos_csv
  TYPE = CSV
  FIELD_DELIMITER = '|'
  SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  NULL_IF = ('', 'NULL', 'null', '\\N')
  DATE_FORMAT = 'YYYY-MM-DD'
  TIMESTAMP_FORMAT = 'YYYY-MM-DD HH24:MI:SS'
  ENCODING = 'UTF8'
  COMPRESSION = AUTO;

-- Sanity-check: list what files are visible in the stage.
LIST @ext_pos_stage/2024/11/;

-- Preview a file without loading (dry run).
SELECT $1, $2, $3, $4, $5
FROM   @ext_pos_stage/2024/11/pos_20241105.csv.gz
(FILE_FORMAT => ff_pos_csv)
LIMIT 10;

-- The actual load.
COPY INTO raw.raw_pos_transactions
  FROM @ext_pos_stage/2024/11/
  FILE_FORMAT = (FORMAT_NAME = ff_pos_csv)
  PATTERN     = '.*pos_20241105\\.csv\\.gz'
  ON_ERROR    = 'SKIP_FILE_10%';   -- skip a file if >10% rows bad
```

sql — Load from a local laptop - the PUT + COPY INTO pattern for one-off files

```
-- From SnowSQL on your laptop:

-- 1. Create a named internal stage.
CREATE OR REPLACE STAGE load_stage
  COMMENT = 'Internal named stage for ad-hoc uploads.';

-- 2. Upload local files to the stage (SnowSQL client-side command).
PUT file:///Users/srikanth/data/products.csv @load_stage
  AUTO_COMPRESS = TRUE   -- gzip on upload (default)
  PARALLEL      = 4;     -- 4 parallel upload threads

-- 3. Verify the file landed.
LIST @load_stage;
```

```
-- 4. Load rows into the target table.
COPY INTO dim_product_raw
  FROM @load_stage/products.csv.gz
  FILE_FORMAT = (TYPE = CSV
                 FIELD_OPTIONALLY_ENCLOSED_BY = '"'
                 SKIP_HEADER = 1)
  ON_ERROR = 'ABORT_STATEMENT';

-- 5. Load history - what happened.
SELECT file_name, status, row_count, error_count, last_load_time
FROM   TABLE(INFORMATION_SCHEMA.LOAD_HISTORY(
           TABLE_NAME => 'DIM_PRODUCT_RAW'))
ORDER BY last_load_time DESC
LIMIT 10;
```

Student

How does data actually get into Snowflake at scale? We saw PUT and COPY INTO briefly in Chapter 1. Now I want the full picture.

Claude • AI Assistant

Data loading in Snowflake follows one canonical pattern regardless of source: **land files in a stage**, then **COPY INTO** a table using a **file format**. That is it.

The stage is where files live; the file format tells Snowflake how to parse them; COPY INTO is the command that moves parsed rows from stage files into the target table. Master this three-part pattern and you can load anything - CSV, JSON, Parquet, Avro, ORC - from anywhere.

STAGES - THE FILE LANDING ZONE

Student

What is a stage, exactly?

Claude • AI Assistant

CONCEPT CARD

A **stage** is a named location in Snowflake that points at files. Files sit in the stage; COPY INTO reads them from there. Four kinds:

User stage (`@~`) - Every user automatically has one. Files uploaded here belong to that user. `PUT file:///Users/srikanth/data/products.csv @~/mango_loads/` . Perfect for personal ad-hoc uploads.

Table stage (`@%tablename`) - Every table has one, automatically. Files here are tied to the table's schema. Handy for one-off loads specific to a single table. `PUT file:///data/products.csv`

`@%dim_product;`

Internal named stage (`@stage_name`) - You create it explicitly with `CREATE STAGE` . Lives in a schema; multiple people and multiple tables can use it.

This is the **standard for internal loads** in a shared environment. `CREATE STAGE mango_mart.retail.load_stage;` then `PUT file:///data/*.csv @mango_mart.retail.load_stage;` .

External stage - Points at YOUR S3 / Azure Blob / GCS bucket. Files stay in your bucket; Snowflake reads them from there.

Requires a **Storage Integration** (secure connection) plus the CREATE STAGE that references it. This is the standard for **enterprise loads** where data lands in your data lake first and Snowflake reads from it.

FILE FORMATS - TELLING SNOWFLAKE HOW TO PARSE

Student

What is a file format?

Claude • AI Assistant

CONCEPT CARD

A **file format** is a named parsing specification. You define it once and reference it in every COPY INTO. Example:

sql — Code Example

```sql
CREATE OR REPLACE FILE FORMAT ff_csv_standard
  TYPE = CSV
  FIELD_DELIMITER = ','
  SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  NULL_IF = ('', 'NULL', 'null');
```

The file format captures: how fields are separated, whether the first row is a header, how strings are quoted, what represents NULL, how dates are formatted, what encoding is used (UTF-8, Latin-1), and how to handle bad rows.

Snowflake supports these file types natively: **CSV, JSON, Parquet, Avro, ORC, XML**. For each, the file format has type-specific options - Parquet does not care about field delimiters; JSON does not care about headers.

Student

Do I need a separate file format for each source?

Claude • AI Assistant

It is a good idea. Different sources often use different conventions (CSV from POS uses `|` as delimiter and no header; CSV from CRM uses `,` with a header).

Rather than passing 10 parameters in every COPY INTO, define one file format per source and reference by name. Reusable, maintainable, testable.

COPY INTO - THE ACTUAL LOAD COMMAND

Student

Now the load command itself.

Claude • AI Assistant

CONCEPT CARD

`COPY INTO <target_table> FROM <stage_path> [FILE_FORMAT = <ff>] [pattern] [options]` is the core loading verb. Its behavior:

- **Reads files from the stage** matching the pattern (or all files if no pattern).
- **Parses each file** using the referenced FILE_FORMAT.
- **Loads rows** into the target table.
- **Tracks loaded files** in metadata so subsequent runs skip already-loaded files. This is Snowflake's built-in idempotency - re-running COPY INTO does not double-load.
- **Returns a load report** - rows loaded, rows rejected, files processed.

Student

How do I select only some files?

Claude • AI Assistant

EXAMPLE CARD

The `PATTERN` option uses a regex against filenames:

sql — Code Example

```sql
COPY INTO dim_product
FROM   @load_stage
FILE_FORMAT = (FORMAT_NAME = ff_csv_standard)
PATTERN = '.*products_.*\.csv\.gz'   -- match files like products_20241105.csv.gz
ON_ERROR = 'CONTINUE';
```

This is how nightly ETL loads look: 'load every file matching today's date pattern.' The regex uses standard POSIX syntax.

THE FULL END-TO-END FLOW

Student

Walk through a realistic Mango Mart load.

Claude • AI Assistant

EXAMPLE CARD

Nightly load of yesterday's POS transactions from the on-prem POS system into Mango Mart:

Step 1 - The POS system exports yesterday's transactions as `pos_20241105.csv.gz` and pushes it to `s3://mango-inbound/pos/2024/11/` .

Step 2 - Snowflake has an external stage pointing at that bucket: `CREATE STAGE ext_pos_stage URL='s3://mango-inbound/pos/' STORAGE_INTEGRATION = si_mango_inbound;` .

Step 3 - Snowflake has a file format for POS CSVs: `CREATE FILE FORMAT ff_pos_csv TYPE=CSV FIELD_DELIMITER='|' SKIP_HEADER=1 ENCODING='UTF8';` .

Step 4 - The nightly ETL job runs on `ETL_WH` :

sql — Code Example

```sql
COPY INTO raw.raw_pos_transactions
FROM   @ext_pos_stage/2024/11/
FILE_FORMAT = (FORMAT_NAME = ff_pos_csv)
PATTERN = '.*pos_20241105\.csv\.gz'
ON_ERROR = 'SKIP_FILE_10%';
```

Step 5 - Downstream SQL transforms `raw_pos_transactions` into fact*sale*stransaction via MERGE. The raw table is TRANSIENT with 0-day time travel; the fact table is PERMANENT with 30-day time travel.

Step 6 - Metadata verification - `SELECT * FROM information_schema.load_history` shows what files loaded, how many rows, how many rejected.

PARQUET VS CSV - WHEN TO PICK WHICH

Student

Should we standardize on CSV or Parquet for the data lake?

Claude • AI Assistant

CONCEPT CARD

For any new data pipeline building in the last few years, **Parquet** is almost always the right choice. Why:

- **Columnar and compressed** - typical 5-10x smaller than equivalent CSV. Storage and network transfer both cheaper.

- **Typed schema** - column types are embedded in the file. Snowflake reads them directly; no `NULL_IF` gymnastics or date-format guessing.
- **Schema evolution** - adding a column to Parquet files does not break existing loads.
- **Column pruning at load time** - Snowflake can load a subset of columns without reading the whole file.
- **Standard across the data ecosystem** - Spark, Databricks, dbt, Airbyte all produce and consume Parquet natively.

CSV is still fine for **one-off loads**, human-inspectable exports, and legacy source systems that only produce CSV. But for anything you build fresh, prefer Parquet.

JSON is the right choice for **semi-structured data** - clickstream events, API responses, IoT payloads. Snowflake's VARIANT type handles JSON beautifully; we cover this in Chapter 3.

SNOWPIPE - THE STREAMING ALTERNATIVE

Student

What if I want data loaded within seconds, not on a nightly batch?

Claude • AI Assistant

`COPY INTO` is the batch loader - you or a scheduler triggers it. For **continuous ingestion**, Snowflake has **Snowpipe**: a service that watches a stage and automatically loads files as they arrive, typically within 1-2 minutes of file drop.

Snowpipe uses the same file format + COPY INTO logic; you just wrap it in a PIPE object that runs continuously.

Snowpipe is billed per file loaded (a very small charge per file), which is a different pricing model from warehouse-per-second. Perfect for streams of many small files. Full coverage in Chapter 3.

ANALOGY

The load pipeline is like **a factory assembly line**. Stage = the loading dock (files arrive here).

File format = the parts specification (tells the line what shape parts to expect). COPY INTO = the assembly step (turns parts into finished product in the warehouse).

File tracking metadata = the production log (ensures the same part is not assembled twice). Snowpipe = the just-in-time delivery version - assembly happens as parts arrive, not in nightly batches.

REAL WORLD APPLICATION

A Chennai logistics client's original ETL took **6 hours** every night. Root cause: they were loading 800 GB of CSV via `COPY INTO` on a MEDIUM warehouse from an internal stage.

Two changes fixed it: (1) their source system was rewritten to produce Parquet instead of CSV, cutting the file size to 90 GB. (2) The load moved to a LARGE warehouse just for the load window (then downsized).

Total load time dropped from 6 hours to **28 minutes**. Same data, same downstream tables.

Format + right-sized compute made the difference.

⚠ WARNING

The `PATTERN` regex is applied to the **entire path relative to the stage**, not just the filename. If your stage points at `s3://bucket/` and files are at `s3://bucket/pos/2024/11/file.csv.gz`, the pattern must match `pos/2024/11/file.csv.gz`, not just `file.csv.gz`.

Use `.*file\.csv\.gz` (leading `.*`) to be safe.

COMMON PITFALL / MISTAKE

Forgetting that COPY INTO tracks loaded files and thinking a second COPY INTO of the same files 'didn't work'. It worked - it correctly skipped already-loaded files.

If you truly want to reload, use `FORCE = TRUE` in the COPY INTO options. But 99% of the time, the skip is what you want.

EXAM TIP CARD

SnowPro Core will test: (1) four stage types and their prefixes (@~, @%table, @stage*name*, @*ext*stage). (2) FILE FORMAT syntax for CSV vs JSON vs Parquet.

(3) COPY INTO idempotency via file tracking. (4) Difference between PUT (upload file to internal stage from your laptop) and COPY INTO (load rows from stage to table).

PRACTICE QUIZ

You want to load new data from S3 into a table without duplicating rows if the pipeline runs twice. What handles this?

A) A UNIQUE constraint on the table

B) MERGE instead of COPY INTO

C) Snowflake's built-in load-history tracking of processed files

D) A pre-load DELETE statement

▼ Reveal Correct Answer

Answer: C. COPY INTO tracks loaded files per stage; a second run skips them by default. No unique constraint or MERGE gymnastics needed for idempotency.

EXERCISE

Design the load pattern for a new Mango Mart source: customer service call logs arriving as Parquet files in `s3://mango-inbound/csr/YYYY/MM/DD/` every hour. Write: (a) the storage integration, (b) the external stage, (c) the file format, (d) the COPY INTO for one day's files.

AI DEEP DIVE PROMPT

Ask an AI: '*Design a Snowflake data loading pipeline for a retail company that receives POS CSVs nightly, e-commerce order JSONs continuously, and inventory Parquet extracts weekly. Include stage, file format, and*

> *COPY INTO for each.*'

SUMMARY CARD

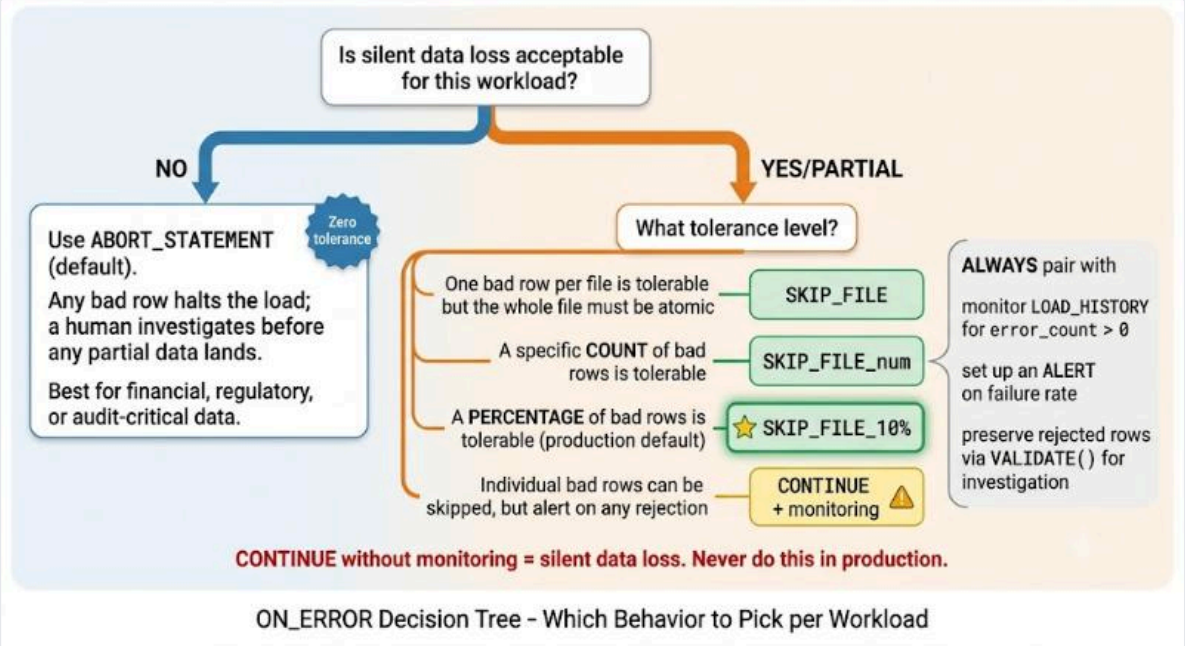
Snowflake loading is a three-part pattern: **stage** (where files live), **file format** (how to parse), **COPY INTO** (load rows). Four stage types: user (`@~`), table (`@%tbl`), internal named (`@stg`), and external (points at your S3/Blob/GCS).

CSV is fine for legacy; **Parquet is preferred** for new pipelines (typed, compressed, columnar). COPY INTO **tracks loaded files** for automatic idempotency.

Snowpipe wraps the same logic for near-real-time continuous loads.

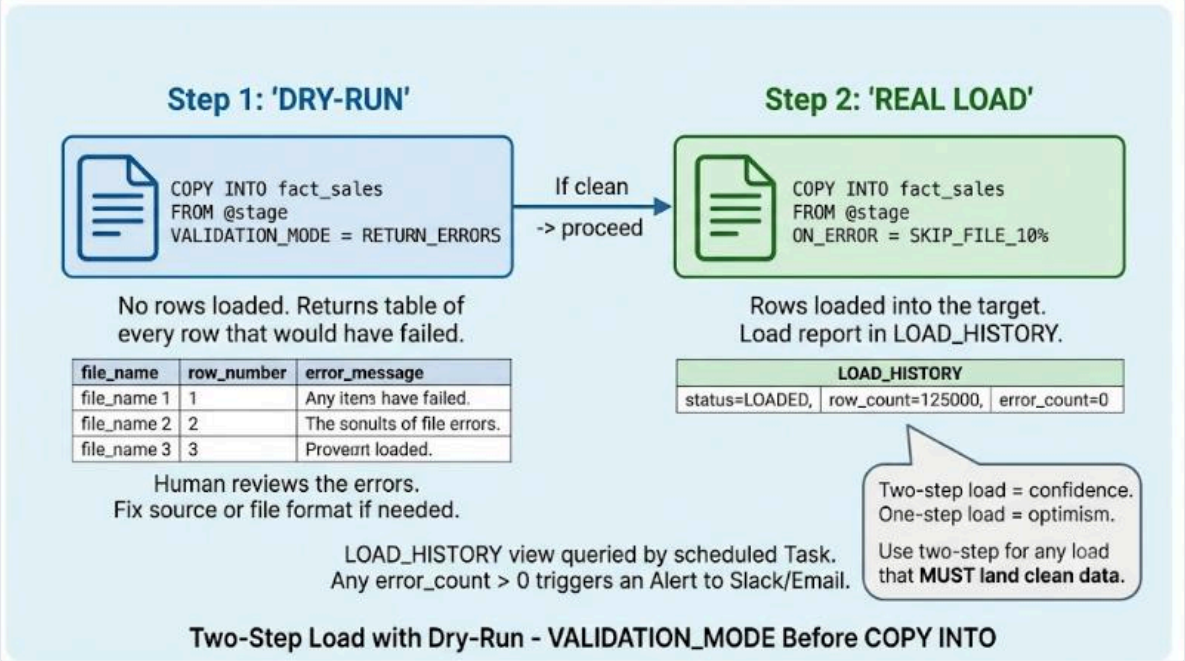
ERROR HANDLING, VALIDATION MODES, AND LOAD MONITORING

Claude • AI Assistant



ON_ERROR Decision Tree - Which Behavior to Pick per Workload

ON_ERROR Decision Tree - Which Behavior to Pick per Workload



Two-Step Load with Dry-Run - VALIDATION_MODE Before COPY INTO

sql — Dry-run first, then load - the safe production pattern

```sql
-- Dry-run: return all errors without loading.
COPY INTO raw.raw_pos_transactions
  FROM @ext_pos_stage/2024/11/pos_20241105.csv.gz
  FILE_FORMAT     = (FORMAT_NAME = ff_pos_csv)
  VALIDATION_MODE = 'RETURN_ERRORS';
-- Review the returned rows. Each has file_name, row_number,
-- category, rejected_record, and the specific error_message.

-- Dry-run: preview the first 10 rows as they would load.
COPY INTO raw.raw_pos_transactions
  FROM @ext_pos_stage/2024/11/pos_20241105.csv.gz
  FILE_FORMAT     = (FORMAT_NAME = ff_pos_csv)
  VALIDATION_MODE = 'RETURN_10_ROWS';

-- If dry-run is clean, do the real load.
COPY INTO raw.raw_pos_transactions
  FROM @ext_pos_stage/2024/11/
  FILE_FORMAT = (FORMAT_NAME = ff_pos_csv)
  PATTERN     = '.*pos_20241105\\.csv\\.gz'
  ON_ERROR    = 'SKIP_FILE_10%';

-- Check what actually happened (14-day retention).
SELECT file_name, status, row_count, error_count,
       first_error_message, last_load_time
FROM   TABLE(INFORMATION_SCHEMA.LOAD_HISTORY(
           TABLE_NAME => 'RAW_POS_TRANSACTIONS'))
WHERE  last_load_time > DATEADD('HOUR', -2, CURRENT_TIMESTAMP)
ORDER  BY last_load_time DESC;

-- Fetch rejected rows from the LAST COPY INTO into this table.
```

```
SELECT *
FROM   TABLE(VALIDATE(raw.raw_pos_transactions, JOB_ID => '_last'));
```

sql — Account-wide load monitoring - detect failures across every table

```
-- Load failures in the last 24 hours across the whole account.
-- ACCOUNT_USAGE has up to 45-minute latency but 365 days of history.
SELECT
    table_catalog_name || '.' || table_schema_name || '.' || table_name
                                                            AS full_table_name,
    file_name,
    status,
    row_count,
    row_parsed,
    error_count,
    first_error_message,
    last_load_time
FROM   snowflake.account_usage.load_history
WHERE  last_load_time > DATEADD('HOUR', -24, CURRENT_TIMESTAMP)
  AND  status IN ('LOAD_FAILED', 'PARTIALLY_LOADED')
ORDER  BY last_load_time DESC;

-- Reliability rollup - success rate per table over the last 7 days.
SELECT
    table_name,
    COUNT(*)                                        AS total_files,
    SUM(CASE WHEN status = 'LOADED' THEN 1 ELSE 0 END) AS successful,
    SUM(CASE WHEN status <> 'LOADED' THEN 1 ELSE 0 END) AS failed_or_partial,
    ROUND(100.0 * SUM(CASE WHEN status = 'LOADED' THEN 1 ELSE 0 END)
               / NULLIF(COUNT(*), 0), 1)              AS success_pct
FROM   snowflake.account_usage.load_history
WHERE  last_load_time > DATEADD('DAY', -7, CURRENT_TIMESTAMP)
GROUP  BY table_name
HAVING failed_or_partial > 0
ORDER  BY failed_or_partial DESC;
```

Student

What happens when things go wrong during a load? Bad rows, mismatched columns, files that fail?

Claude • AI Assistant

Error handling is where beginners and professionals part ways. A production load pipeline must decide, in advance, what happens when a row fails to parse, when a whole file is malformed, when the load is halfway through a batch.

Snowflake gives you five behaviors on the COPY INTO command and two validation modes that let you dry-run a load before committing. This subsection makes you fluent in both.

THE ON_ERROR OPTIONS

Student

Walk me through the five behaviors.

Claude • AI Assistant

CONCEPT CARD

`ON_ERROR` is a COPY INTO option that decides what happens when a row fails to parse or violates a type constraint. Five values:

ABORT_STATEMENT (default) - the entire COPY INTO aborts on the first error. No rows loaded. Strictest option; safest for production data that must be atomic.

CONTINUE - skip the bad row silently, keep loading the rest of the file. Loosest option. Good for exploration but dangerous in production because a rejected row disappears without alerting anyone.

SKIP_FILE - abandon the whole file at the first bad row. Other files in the batch continue. Useful when you want file-grain atomicity.

SKIP_FILE_num - abandon the file if it accumulates `num` bad rows. `SKIP_FILE_100` skips only if the file has 100+ bad rows - tolerates a small number.

SKIP_FILE_num% - abandon the file if bad rows exceed `num` % of the file's total. `SKIP_FILE_10%` is the production default - tolerate up to 10% bad rows before declaring the file unusable.

Student

Which should I use?

Claude • AI Assistant

Match the option to your data quality expectations:

- **Regulatory / financial reporting** - `ABORT_STATEMENT` . Any bad row halts the pipeline; a human must investigate.
- **Analytics dashboards** - `SKIP_FILE_10%` . Small format glitches shouldn't kill a whole load, but if 10% of a file is bad the file is broken and should be re-generated at source.
- **Log data / clickstream** - `CONTINUE` with a monitoring alert on rejected rows. High volume; small percentage of noise expected.
- **Development / exploration** - `CONTINUE` to see what data actually loads, then investigate rejections.

VALIDATION MODES - DRY-RUN BEFORE YOU LOAD

Student

Can I preview what will happen before actually loading?

Claude • AI Assistant

CONCEPT CARD

Yes - COPY INTO has a `VALIDATION_MODE` option that runs the parse and type-check without writing rows to the target table. Three modes:

RETURN_ERRORS - returns a list of every parse/type error in the source files. No rows loaded. Perfect for QA before a big load.

RETURN_num_ROWS - returns the first `num` rows that would be loaded (parsed and typed). Lets you visually confirm the parse is correct.

RETURN_ALL_ERRORS - like RETURN*ERRORS but includes rows that would have been skipped by the ON*ERROR setting. Comprehensive audit of any row that had any issue.

Example workflow: run COPY INTO with `VALIDATION_MODE = 'RETURN_ERRORS'` first; inspect the report; fix the source or the file format if needed; run again with the validation mode removed to actually load. Two-step but bulletproof.

LOAD HISTORY AND MONITORING

Student

After a load runs, how do I see what happened?

Claude • AI Assistant

CONCEPT CARD

Two views expose load history:

`INFORMATION_SCHEMA.LOAD_HISTORY` - low-latency, scoped to the current database. Shows the last 14 days of loads for one table via `TABLE(INFORMATION_SCHEMA.LOAD_HISTORY(TABLE_NAME => 'FACT_SALES'))`. This is what you use in the same session as a load to check the result.

`SNOWFLAKE.ACCOUNT_USAGE.LOAD_HISTORY` - account-wide, up to 365 days of history, but with 45-minute latency. Use this for dashboards showing load reliability across all your tables.

Both give you per-file details: file name, row count, error count, status ('LOADED', 'PARTIALLY*LOADED*', '*LOAD*FAILED'), and the specific error messages if failures occurred.

Student

How do I set up alerting on load failures?

Claude • AI Assistant

EXAMPLE CARD

The standard pattern is a scheduled Snowflake **Task** (see Chapter 4) that queries LOAD_HISTORY for any file with status != 'LOADED' in the last hour and sends a notification via a Snowflake **Alert** (a native alerting primitive) or via an integration to Slack/PagerDuty. Skeleton:

sql — Code Example

```sql
CREATE OR REPLACE ALERT alert_load_failures
  WAREHOUSE = load_wh
  SCHEDULE  = '15 MINUTE'
  IF (EXISTS (
    SELECT 1 FROM information_schema.load_history
    WHERE status IN ('LOAD_FAILED','PARTIALLY_LOADED')
      AND last_load_time > DATEADD(minute, -15, CURRENT_TIMESTAMP)
  ))
  THEN CALL system$send_email(...);
```

REJECTED ROWS - WHERE THEY GO

Student

If I use CONTINUE or SKIP*FILE*10%, where do the rejected rows go? Are they lost?

Claude • AI Assistant

CONCEPT CARD

By default, rejected rows are **logged in LOAD_HISTORY** with a reason but not preserved as data. If you want the actual rejected row content preserved, use the `VALIDATION_MODE` pattern above (dry-run before real load) or add a **staging table for rejects** to your pipeline pattern:

1. COPY INTO the target table with `ON_ERROR = 'CONTINUE'` .
2. Then run: `SELECT * FROM TABLE(VALIDATE(target_table, JOB_ID => '_last')) ...` - this returns rows that failed the LAST COPY INTO into that table, including their raw data.
3. Insert those rows into a rejects table for investigation.

This is the standard pattern for pipelines that must tolerate some errors but preserve failed rows for later review.

FILE-LEVEL IDEMPOTENCY AND FORCE

Student

What if I need to actually reload a file that was already processed?

Claude • AI Assistant

CONCEPT CARD

By default, COPY INTO tracks loaded file names for **64 days** and skips them on re-runs. This is your idempotency guarantee - a nightly job that reruns doesn't double-load. To override:

- `FORCE = TRUE` - reload files even if already loaded. Use surgically - it can produce duplicates.
- `PURGE = TRUE` - delete files from the stage after successful load. Use when the stage is a landing zone that should not accumulate files forever.

For a full reprocess (rare but sometimes needed for data corrections): TRUNCATE the target table, then COPY INTO with `FORCE = TRUE` to reload everything.

ANALOGY

The ON*ERROR options are like **assembly line quality control policies**. ABORT*STATEMENT is 'halt the line at the first defect' - strict factories.

CONTINUE is 'let the defective units through and count them' - loose, high-volume production. SKIP*FILE*10% is 'reject a batch if more than 10% is bad' - the industry standard for most manufacturers.

Choose based on the cost of a defect reaching the customer.

REAL WORLD APPLICATION

A payments client's data engineering team used `ON_ERROR = 'CONTINUE'` on their transaction loads. For six months.

During a routine audit, they discovered they had silently dropped 2.4 million rows over that period - all with format errors that ON_ERROR = CONTINUE had swallowed. The rows represented $84 million in transaction volume that never made it to the warehouse.

Fix: (a) changed to `ABORT_STATEMENT` in production, (b) added a monitoring alert on any rejected-row count > 0. Since then, zero silent losses.

The story is a permanent slide in their onboarding deck.

WARNING

Never use `ON_ERROR = 'CONTINUE'` in production without an accompanying alert on rejected rows. Silent data loss is the worst kind of data quality problem - by the time anyone notices, the damage has been compounding for weeks.

COMMON PITFALL / MISTAKE

Assuming that a successful COPY INTO means every row loaded. It means the COPY INTO command succeeded per the ON*ERROR setting. With CONTINUE, some rows may have been skipped. Always check LOAD*HISTORY for `error_count > 0` after every load.

EXAM TIP CARD

SnowPro Core loves ON*ERROR variants and validation modes. Memorize: ABORT*STATEMENT is the default; SKIP*FILE*10% is percentage-based; VALIDATION*MODE = RETURN*ERRORS does not load rows. Also: file tracking metadata retains loaded filenames for 64 days by default.

PRACTICE QUIZ

You are loading a nightly file of credit card transactions. If any row fails to parse, you want the entire load to abort so a human can investigate. Which ON_ERROR setting?

A) CONTINUE B) SKIP*FILE*10% C) ABORT*STATEMENT D) SKIP*FILE_100

▼ Reveal Correct Answer

Answer: C. ABORT_STATEMENT - any error halts the load. Financial data must be atomic; silent skipping is unacceptable.

EXERCISE

Design the ON_ERROR strategy for three Mango Mart loads: (a) hourly clickstream from the e-commerce website (millions of rows, small percentage of malformed events expected), (b) nightly finance ledger from SAP (must be atomic - any error aborts), (c) weekly supplier catalog updates (small file, want to see which rows failed for manual correction).

AI DEEP DIVE PROMPT

Ask an AI: '*Design a production data quality strategy for a Snowflake load pipeline with three data sources of different criticality levels. Include ON_ERROR settings, validation mode usage, and monitoring for silent row rejections.*'

SUMMARY CARD

Every COPY INTO must choose an **ON_ERROR** behavior - ABORT*STATEMENT (default, strict), CONTINUE (silent skip - dangerous alone), SKIP*FILE*num% (percentage-tolerant, the production standard).* **VALIDATION*MODE** lets you dry-run a load before committing.

LOAD_HISTORY shows per-file results with 14-day (INFORMATION*SCHEMA) or 365-day (ACCOUNT*USAGE) visibility. **File tracking** prevents accidental double-loads.

And critically: never use CONTINUE without an alert on rejected-row counts - silent data loss is the worst failure mode.

CHAPTER REVIEW & SUMMARY

Claude • AI Assistant

sql — Chapter 2 in a single snapshot - one runnable block covering the whole chapter

```sql
-- Everything Chapter 2 taught, distilled into a runnable block.

USE ROLE sysadmin;
USE WAREHOUSE compute_wh;

-- 1. Databases and table types (2.4)
```

```
CREATE DATABASE IF NOT EXISTS mango_mart;
CREATE SCHEMA IF NOT EXISTS mango_mart.raw;
CREATE SCHEMA IF NOT EXISTS mango_mart.analytics;

-- Raw staging: transient, no fail-safe, minimal time travel
CREATE OR REPLACE TRANSIENT TABLE mango_mart.raw.raw_pos_transactions (
  txn_id VARCHAR(25), sale_ts TIMESTAMP_NTZ,
  store_code VARCHAR(10), product_code VARCHAR(20),
  qty INT, unit_price NUMBER(10,2), net_revenue NUMBER(12,2)
) DATA_RETENTION_TIME_IN_DAYS = 0;

-- Analytics: permanent, 30-day time travel
CREATE OR REPLACE TABLE mango_mart.analytics.fact_sales_transaction (
  date_key INT NOT NULL, store_key INT NOT NULL,
  product_key INT NOT NULL, quantity_sold INT NOT NULL,
  net_revenue NUMBER(12,2) NOT NULL,
  load_date DATE DEFAULT CURRENT_DATE,
  correction_flag TINYINT DEFAULT 0
) DATA_RETENTION_TIME_IN_DAYS = 30;

-- 2. Warehouses (2.3) - workload isolation
CREATE OR REPLACE WAREHOUSE etl_wh WAREHOUSE_SIZE='LARGE'
  AUTO_SUSPEND=60 AUTO_RESUME=TRUE;
CREATE OR REPLACE WAREHOUSE bi_wh WAREHOUSE_SIZE='MEDIUM'
  AUTO_SUSPEND=600 AUTO_RESUME=TRUE
  MIN_CLUSTER_COUNT=1 MAX_CLUSTER_COUNT=4;

-- 3. Stage and file format (2.5)
CREATE OR REPLACE STAGE mango_mart.raw.load_stage;
CREATE OR REPLACE FILE FORMAT mango_mart.raw.ff_csv
  TYPE=CSV FIELD_DELIMITER=',' SKIP_HEADER=1
  FIELD_OPTIONALLY_ENCLOSED_BY='"';

-- 4. COPY INTO with error handling (2.5 + 2.6)
COPY INTO mango_mart.raw.raw_pos_transactions
  FROM @mango_mart.raw.load_stage
  FILE_FORMAT=(FORMAT_NAME=mango_mart.raw.ff_csv)
  ON_ERROR='SKIP_FILE_10%';

-- 5. Zero-copy clone for testing (2.4)
CREATE OR REPLACE TABLE mango_mart.raw.raw_pos_transactions_test
  CLONE mango_mart.raw.raw_pos_transactions;

-- 6. Verify - load history (2.6) and pruning stats (2.2)
SELECT file_name, status, row_count, error_count
FROM   TABLE(INFORMATION_SCHEMA.LOAD_HISTORY(
  TABLE_NAME=>'RAW_POS_TRANSACTIONS'))
ORDER BY last_load_time DESC LIMIT 5;

SELECT query_id, partitions_scanned, partitions_total,
       ROUND(100.0*partitions_scanned/NULLIF(partitions_total,0),1) AS pct
```

```
FROM   TABLE(INFORMATION_SCHEMA.QUERY_HISTORY(RESULT_LIMIT=>5))
ORDER BY start_time DESC;
```

CHAPTER SUMMARY

Chapter 2 took the architectural sketch from Chapter 1 and turned it into operational knowledge. You now understand how a query actually flows through Snowflake's three layers, how micro-partitions and pruning determine performance, how to size warehouses correctly, how the four table types differ, and how the stage-format-COPY-INTO pattern loads any data from anywhere.

Six ideas.

THE SIX FOUNDATIONAL IDEAS

1. Three-Layer Architecture (2.1). Cloud services (brain: auth, metadata, optimizer, transactions), query processing (muscle: virtual warehouses executing SQL), and storage (memory: encrypted, columnar, immutable micro-partitions on S3/Blob/GCS).

Every query passes through all three. Services is free up to 10% of compute; storage is billed per TB per month; compute is billed per second per warehouse.

2. Micro-Partitions and Pruning (2.2). Snowflake stores tables as automatic 16 MB-compressed columnar files with MIN/MAX metadata per column.

Partition pruning skips partitions that cannot match a WHERE clause - the #1 performance lever. Sargable predicates (no functions on filtered columns) and natural or explicit clustering are how you get maximum pruning.

Query Profile shows partitions*scanned / partitions*total - your primary tuning signal.

3. Virtual Warehouse Sizing (2.3). Sizes double in cost and compute at each step (XS=1 to 6XL=512 credits/hour).

Scale UP for heavy queries, **scale OUT** (multi-cluster) for concurrency. **AUTO_SUSPEND** trades cache warmth for idle-time savings - typical values are 60s for ETL, 600s for BI.

60-second minimum billing on resume is the biggest source of surprise costs. Standard pattern: **one warehouse per workload**, not per team.

4. Database, Schema, and Table Types (2.4). Databases separate broad domains (RAW, STAGING, ANALYTICS).

Four table types: **permanent** (default, full Time Travel + Fail-Safe), **transient** (no Fail-Safe, lower cost - use for staging), **temporary** (session-scoped, cheapest), **external** (data in your own S3/Blob/GCS). **Zero-copy cloning** creates writable snapshots in seconds without duplicating storage - the killer feature for dev/test.

5. Data Loading (2.5). The universal pattern is **stage -> file format -> COPY INTO**.

Four stage types (user @~, table @%tbl, internal named @stg, external). **Parquet** is preferred for new pipelines (typed, compressed, columnar); CSV for legacy.

COPY INTO **tracks loaded files** for automatic idempotency. **Snowpipe** wraps the same logic for continuous loads (Chapter 3 goes deeper).

6. Error Handling and Monitoring (2.6). Every COPY INTO chooses an **ON_ERROR** behavior.

ABORT*STATEMENT (default, strict) for regulatory data; SKIP*FILE*10% for typical analytics loads; CONTINUE only with monitoring on rejected rows. **VALIDATION*MODE** lets you dry-run before committing.

LOAD_HISTORY shows what actually loaded. Never use CONTINUE without an alert on rejected-row counts - silent data loss is the worst failure mode.

ONE-SENTENCE CHAPTER DISTILLATION

Snowflake's **three layers** independently scale services (brain), compute (muscle), and storage (memory); every table is stored as **automatic columnar micro-partitions** whose metadata drives **pruning**; you **right-size warehouses per workload** and load data via the universal **stage -> format -> COPY INTO** pattern with explicit **error handling** and **load history** monitoring.

END-OF-CHAPTER CAPSTONE

EXERCISE

Complete all six steps end-to-end. Timebox: 2 hours.

1. **Create two databases** in your Mango Mart account: `RAW` (transient tables, 0-day time travel) and `ANALYTICS` (permanent tables, 30-day time travel).
2. **Create three warehouses** with the standard pattern: `ETL_WH` (LARGE, AUTO*SUSPEND=60), `BI*WH `(MEDIUM, multi-cluster 1-4, AUTO_SUSPEND=600), LOAD*WH` (XSMALL, AUTO*SUSPEND=60).
3. **Create a file format** for CSV and one for Parquet in the RAW schema.
4. **Load one CSV file** of sample retail data into `raw.raw_pos_transactions` using PUT (from your laptop) + COPY INTO (with SKIP*FILE*10%).
5. **Verify the load** by querying `INFORMATION_SCHEMA.LOAD_HISTORY` - confirm the file loaded, count rows.
6. **Clone your loaded table** to a test copy: `CREATE TABLE raw_pos_transactions_test CLONE raw_pos_transactions;` . Verify it appears instantly regardless of source size.

If all six steps pass, you have exercised every mechanism in Chapter 2 - warehouses, databases, table types, stages, file formats, COPY INTO, load history, and cloning. Chapter 3 (Time Travel, Snowpipe, Clustering, RBAC, Semi-structured data) builds directly on this base.

VOCABULARY YOU SHOULD OWN BY NOW

Cloud services layer, query processing layer, storage layer, virtual warehouse, warehouse size (XS to 6XL), AUTO*SUSPEND, AUTO*RESUME, MIN/MAX*CLUSTER*COUNT, SCALING*POLICY (STANDARD vs ECONOMY), micro-partition, columnar storage, MIN/MAX metadata, partition pruning, sargable predicate, clustering key, permanent/transient/temporary/external table, DATA*RETENTION*TIME*IN*DAYS, zero-copy cloning, UNDROP, stage (user @~, table @%tbl, internal named, external), storage integration, file format (CSV, JSON, Parquet), COPY INTO, PATTERN, ON*ERROR (ABORT*STATEMENT, CONTINUE, SKIP*FILE*num%), VALIDATION*MODE, LOAD_HISTORY, VALIDATE, FORCE, PURGE.

SNOWPRO CORE ALIGNMENT

This chapter contributes to three SnowPro Core exam domains:

- **Snowflake Data Cloud Features and Architecture** - three-layer architecture, micro-partitions, cloning.
- **Virtual Warehouses** - sizing, auto-suspend, multi-cluster, credit consumption.

- **Data Loading and Unloading** - stages, file formats, COPY INTO, ON*ERROR, VALIDATION*MODE.

Expect 8-12 questions on the exam that draw directly from Chapter 2 material - the largest single-chapter contribution in this refcard.

WHAT'S NEXT

In **Chapter 3 - Snowflake Advanced**, we open the features that separate Snowflake practitioners from Snowflake experts: **Snowpipe** for continuous loading, **Time Travel + Zero-Copy Cloning + Fail-Safe** for data recovery and dev/test workflows, **Performance Tuning** (clustering keys, Query Profile, materialized views), **RBAC and Data Masking** for enterprise security, and **Semi-Structured Data** (VARIANT type and FLATTEN) for JSON-heavy workloads. The vocabulary from Chapter 2 - micro-partitions, warehouses, stages - becomes the working currency of every new feature.

SUMMARY CARD

Chapter 2 turned the architectural intuition of Chapter 1 into operational fluency: how the three layers actually process a query, how micro-partitions and pruning determine performance, how warehouse sizing and auto-suspend interact with cost, how the four table types serve different data lifecycles, how the stage-format-COPY-INTO pattern loads anything, and how to handle errors and monitor loads in production. Every subsequent chapter of this refcard builds on these six ideas.